



Generative AI

(updated on 25 Sep 2026)

Professor Peerapon Vateekul, Ph.D.

peerapon.v@chula.ac.th

Department of Computer Engineering,
Faculty of Engineering, Chulalongkorn University

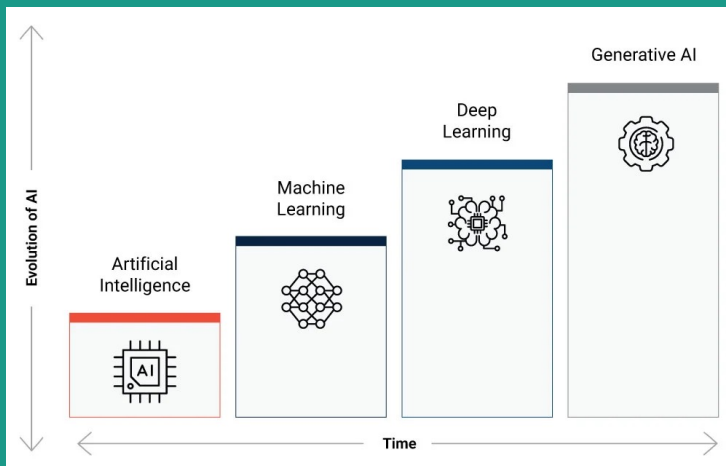


Outline

1. Generative AI & Modern LLMs
2. Prompt Engineering
3. Retrieval-Augmented Generation (RAG)
4. Fine-Tuning (briefly)
5. Tool Use & Agentic AI
6. Context Engineering, Evaluation & Safety, Harness Engineering
7. LLM Tools

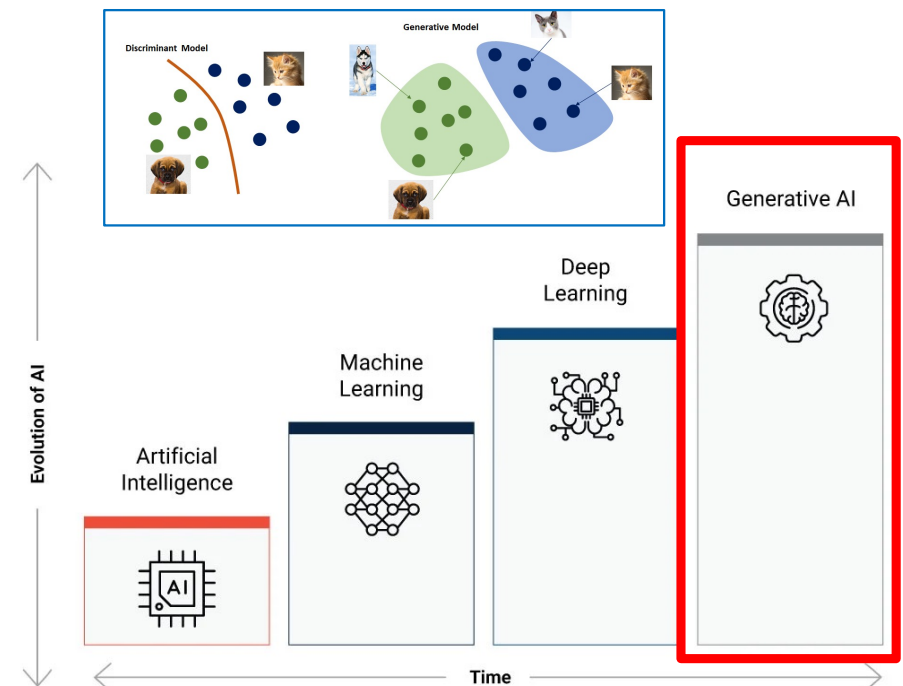
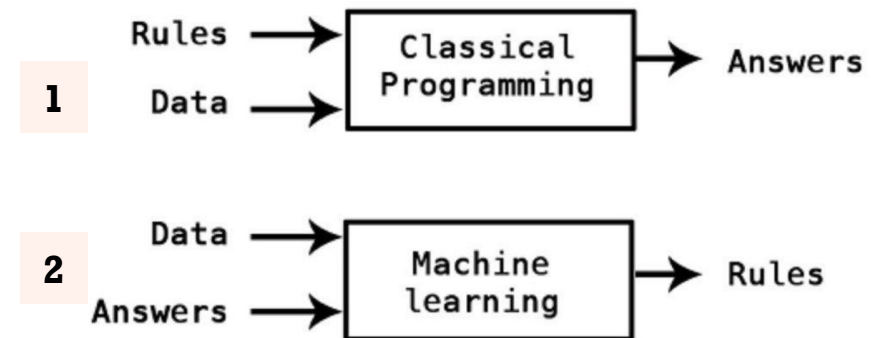
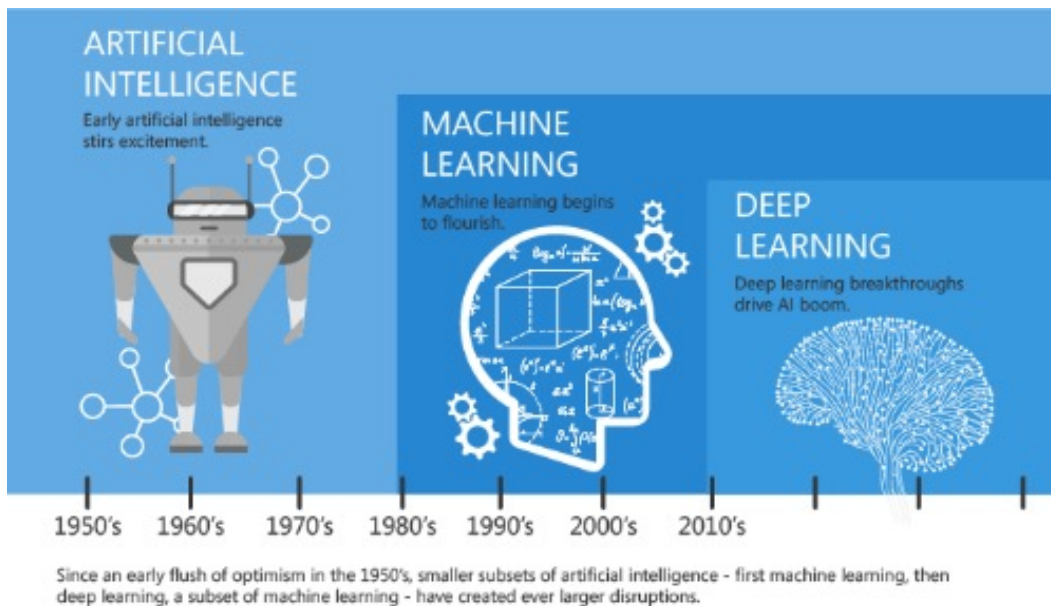
1) Generative AI & Modern LLMs

Credit to Asst. Prof.Ekapol's slide



AI = Automation

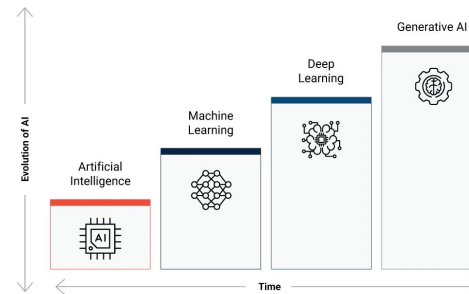
- 1) Rule-based AI
- 2) Machine Learning (ML)



<https://mc.ai/machine-learning-basics-artificial-intelligence-machine-learning-and-deep-learning/>

Past to Present

- Regression (few)
- NN (thousands)
- DL (millions)
- Gen AI (billions)



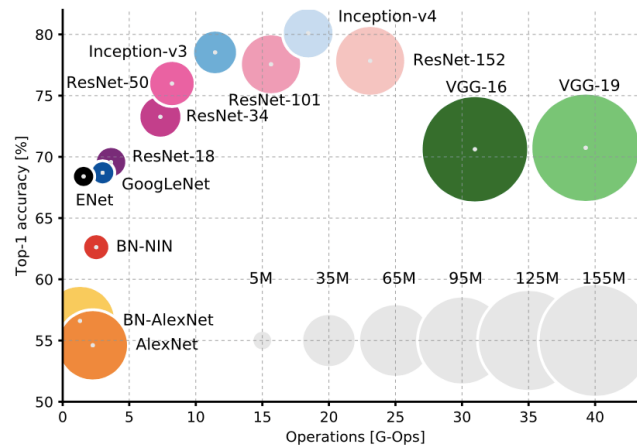
$$\text{Spend} = 500 + 10 \cdot \text{Age} + 20 \cdot \text{Income1K}$$

BASIC REGRESSION

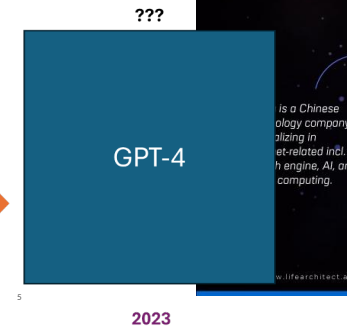
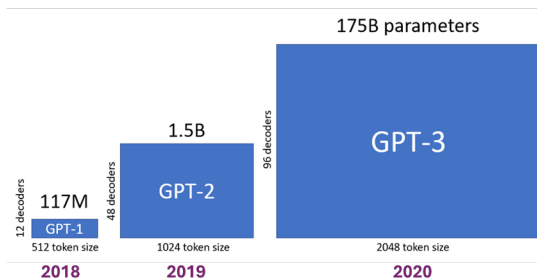
- LINEAR** `linear_model.LinearRegression()`
Lots of numerical data
- LOGISTIC** `linear_model.LogisticRegression()`
Target variable is categorical

CLASSIFICATION

- NEURAL NET** `neural_network.MLPClassifier()`
Complex relationships. Prone to overfitting. Basically magic.
- K-NN** `neighbors.KNeighborsClassifier()`
Group membership based on proximity
- DECISION TREE** `tree.DecisionTreeClassifier()`
If/then/else. Non-contiguous data. Can also be regression
- RANDOM FOREST** `ensemble.RandomForestClassifier()`
Find best split randomly. Can also be regression
- SVM** `svm.SVC()` `svm.LinearSVC()`
Maximum margin classifier. Fundamental Data Science algorithm
- NAIVE BAYES** `GaussianNB()` `MultinomialNB()` `BernoulliNB()`
Updating knowledge step by step with new info



How exactly “Large” it is



Predictive AI vs. Generative AI

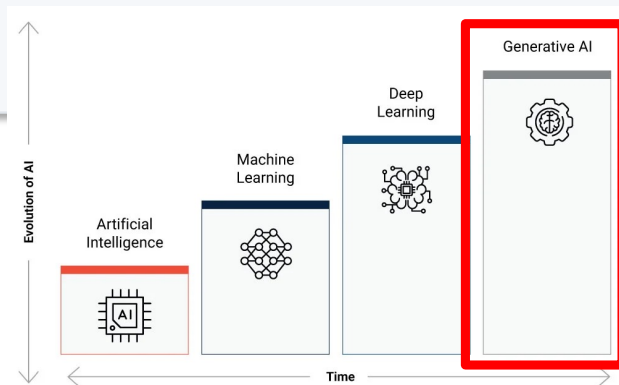
Predictive AI

Input

Model

Prediction

- Classification
- Regression / forecasting
- Detection / segmentation



Generative AI

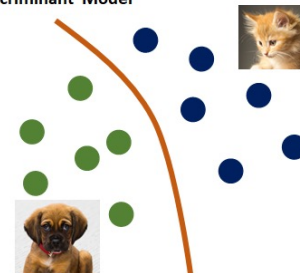
Prompt

Foundation Model

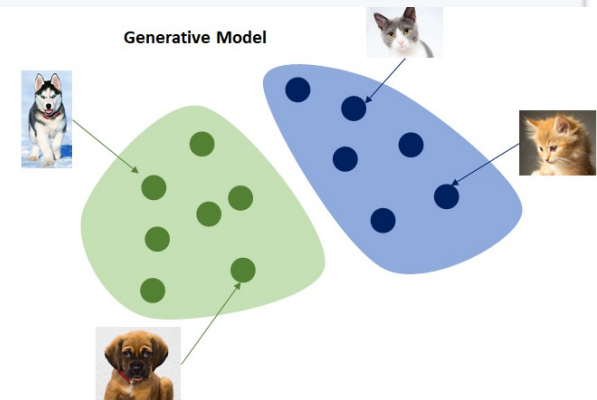
New Content

- Text & code
- Images / audio / video
- Structured outputs

Discriminant Model



Generative Model



ChatGPT

- ChatGPT was launched by OpenAI on 30 Nov 2022.
- ChatGPT is a [large language model \(LLM\)](#) for conversational AI applications.
- Generates human-like text and performs NLP tasks.
- Scalable and flexible for various use cases.
- ChatGPT didn't disclose the details.



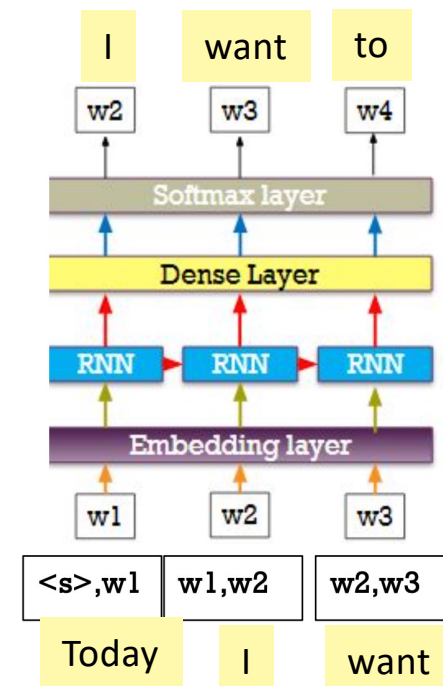
Language Model (LM)

- It is the model that aims to predict next word based on the given previous words.
- So, the model can understand grammar & context.

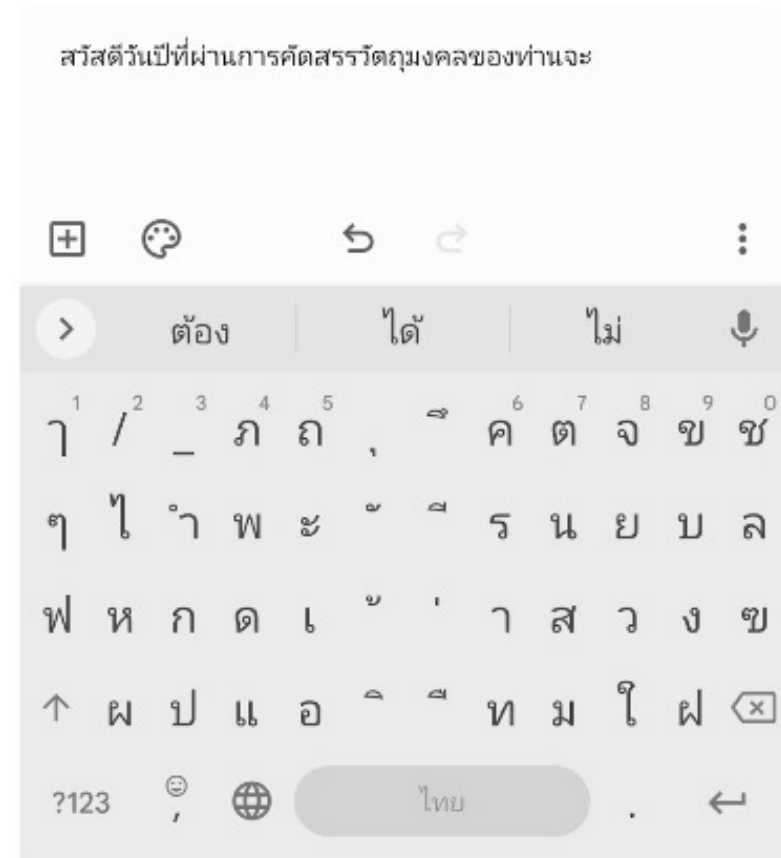
วันนี้เราอยากจะกินข้าวมัน_____

Today I want to eat chick _____ .

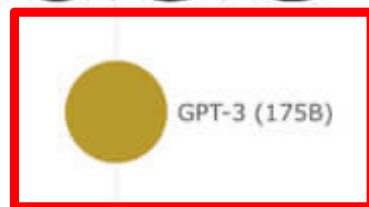
วัน นี้ เรา อยาก จะ กิน ข้าว มัน



You have been using LM 😊 - Autocomplete



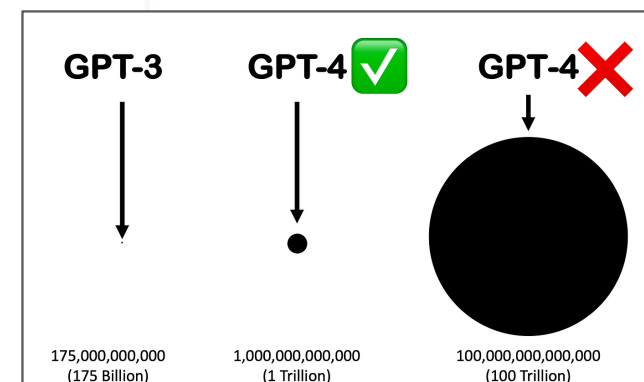
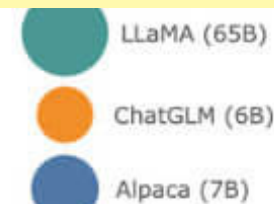
Top Large Language Models



Obama-RNN [2015]
Data: 730,895 tokens (4MB)
Model: 3MB parameters



GPT3 [2020]
Data: ~750GB (30,000x)
Model: 175B parameters (700,000x)
Training cost: \$5M, equivalent to ~300 years



<https://cobusgreyling.medium.com/what-are-realistic-gpt-4-size-expectations-73f00c39b832>

2023



<https://vectara.com/top-large-language-models-llms-gpt-4-llama-gato-bloom-and-when-to-choose-one-over-the-other/>

Step 1

Collect demonstration data and train a supervised policy.

LM

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3.5 with supervised learning.



Step 2

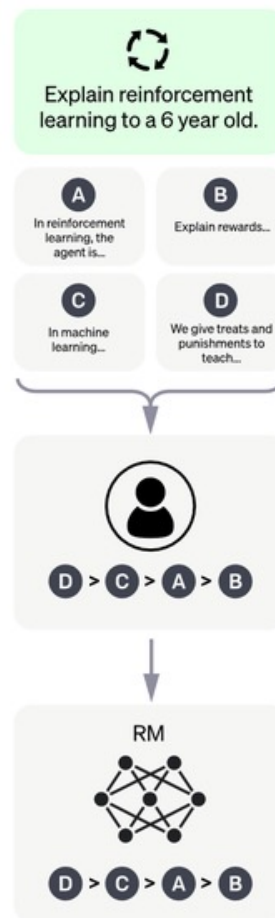
Collect comparison data and train a reward model.

Reward Model (Scoring Model)

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using the PPO reinforcement learning algorithm.

RL

A new prompt is sampled from the dataset.

The PPO model is initialized from the supervised policy.

The PPO model calculates a reward for the output.

The reward is used to update the policy using PPO.



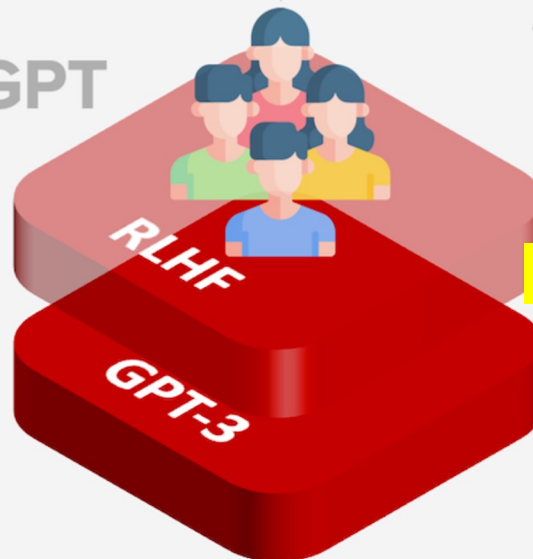
OpenAI



+RL

InstructGPT

Jan/2022



ChatGPT

Nov/2022

+Safety policy



2022. LifeArchitect.ai

Large + LM = LLM

"Large" is no longer just about parameter count.

Model	Year	Parameters	Key advancement
GPT-1	2018	117M	Transformer language modeling
GPT-2	2019	1.5B	Stronger zero-shot generation
GPT-3	2020	175B	In-context & few-shot learning
GPT-4	2023	ND (~1.7TB)	Multimodality + stronger reasoning
GPT-5	2025	ND (~300B)	Integrated reasoning + tool use
GPT-6 Astra	2026	Not disclosed (ND)	Computer use + end-to-end agentic work

Modern LLM capability ≠ parameter count alone

Pretraining + Architecture + Post-training +
Inference-time Reasoning + Tools

Note: OpenAI has not publicly disclosed parameter counts for GPT-4, GPT-5, or GPT-6.

Evolution of LLMs

From language models to agentic models



GPT-6 Astra (2026)

Reasoning • Computer Use • Tools • 1.05M
Context

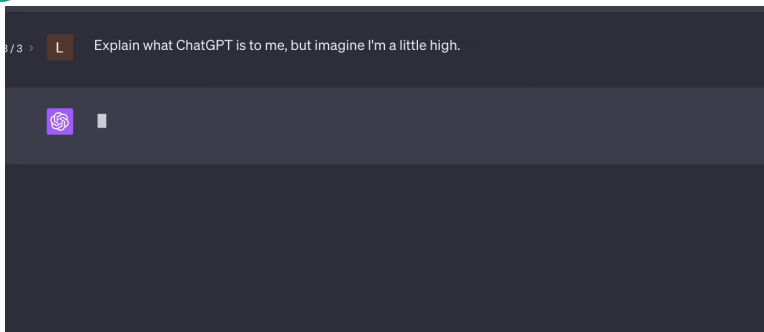
Source: OpenAI official model documentation, Sep 2026

LLM Capabilities

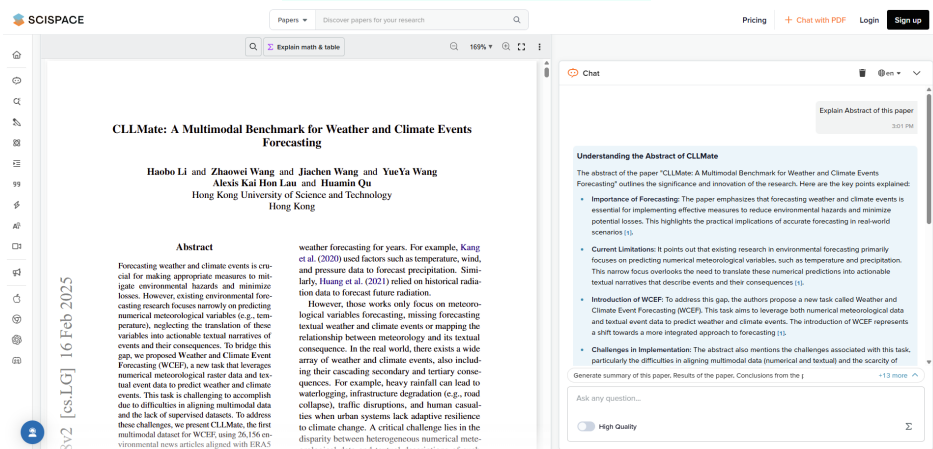


ChatGPT

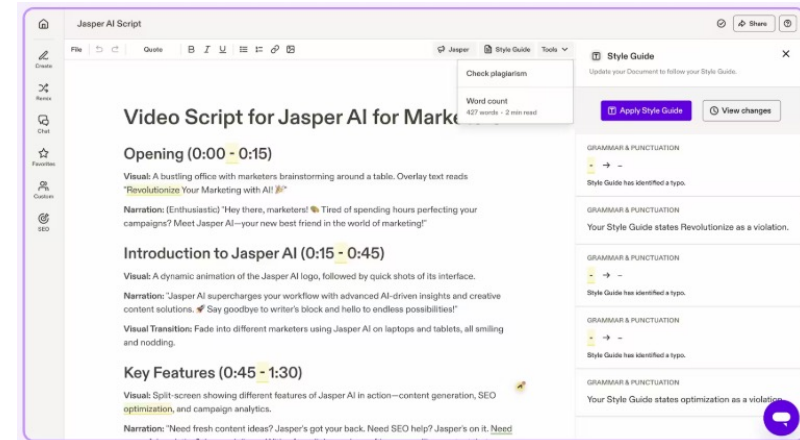
1) Assistant Chatbot



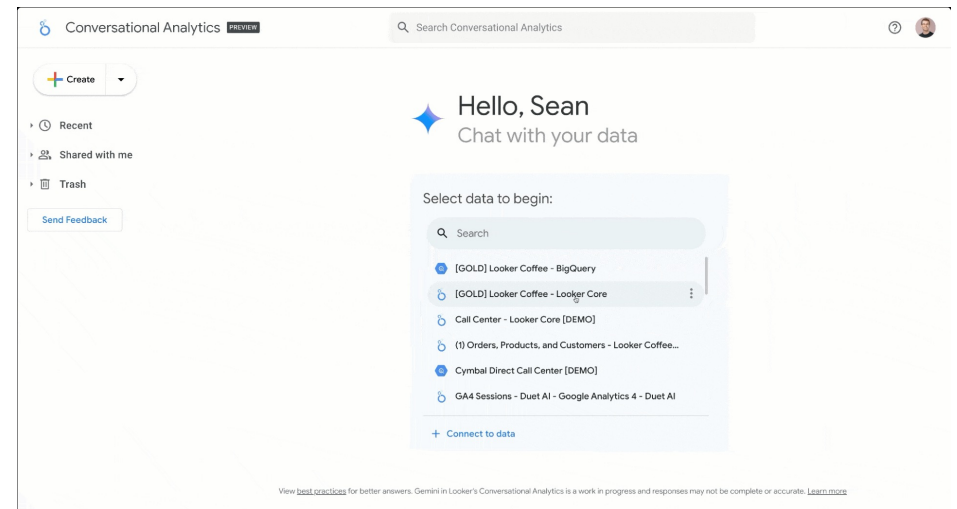
2) Text Summarization



3) Content Creation



4) Data Analytics



Modern LLMs in 2026 – Beyond Text Generation

2026 Trend: From specialized LLMs → unified multimodal, reasoning, and agentic models
Text → Multimodal → Reasoning → Tools → Agents

Category	Capabilities	Examples
1) Frontier / General-Purpose Models	Multimodal + reasoning + tools	GPT-6 Astra, Gemini 3.5, Claude 4.6
2) Reasoning Models	Complex reasoning, math, science, planning	GPT-6, Gemini 3.5, DeepSeek-R1/V-series
3) Multimodal / Omni Models	Text, image, audio, video understanding & generation	Gemini Omni, GPT-6, Gemini 3.5
4) Coding Models	Code generation, debugging, software engineering	GPT-6/Codex, Gemini 3.5, Claude 4.6
5) Agentic Models	Tool use, planning, computer/browser use, long-horizon tasks	GPT-6 Astra, Gemini 3.5, Claude 4.6
6) Long-Context Models	~1M-token context, long documents/codebases	GPT-6 Astra, Claude 4.6, Gemini family
7) Voice / Real-Time Models	Real-time speech, transcription, conversational voice	Gemini 3.5 Transcribe, real-time voice models



List of Interesting LLM models (as of 25 Sep 2026)

1. OpenAI: GPT
2. Anthropic: Claude
3. Google: Gemini
4. Alibaba: Qwen
5. ByteDance: Deepseek

1) OpenAI: GPT Models

Aspect	GPT-5.6 Sol	GPT-5.6 Terra	GPT-5.6 Luna	(New!) GPT-6 Astra 🚀
Positioning	Flagship for complex work	Balanced everyday model	Fast & low-cost	Most capable; hardest end-to-end work
Best for	Reasoning · Coding · Professional work	General workloads	High-volume / cost-sensitive tasks	Reasoning · Coding · Computer Use · Research
Reasoning	none → max	none → max	none → max	low → max
Context	1.05M	1.05M	1.05M	1.05M
Max output	128K	128K	128K	128K
API Price (Input / Output per MTok)	\$4 / \$20	\$2 / \$12	\$0.20 / \$1.20	\$10 / \$50
Launch	Jul 2026	Jul 2026	Jul 2026	Sep 3, 2026

2) Anthropic: Claude Models



New!

Claude Fable 5.1

For demanding reasoning and long-horizon agentic work



Claude Opus 5

For complex agentic coding and enterprise work



Claude Sonnet 5

The best combination of speed and intelligence



Claude Haiku 4.5

The fastest model with near-frontier intelligence

Comparative latency ⓘ

Slower

Moderate

Fast

Fastest

Pricing ⓘ

\$10 / input MTok
\$50 / output MTok

\$5 / input MTok
\$25 / output MTok

\$2 / input MTok
\$10 / output MTok

\$1 / input MTok
\$5 / output MTok

Claude API ID ⓘ

claude-fable-5-1

claude-opus-5

claude-sonnet-5

claude-haiku-4-5-20251001

Capabilities

Thinking ⓘ

Adaptive (always on)

Adaptive

Adaptive

Extended

Default effort ⓘ

high

high

high

Not supported

Context window ⓘ

1M tokens

1M tokens

1M tokens

200K tokens

Max output ⓘ

128K tokens

128K tokens

128K tokens

64K tokens

Reliable knowledge cutoff ⓘ

Jun 2026

May 2026

Jan 2026

Feb 2025

3) Google: Gemini Models

	Gemini 3.1 Pro	(New!) Gemini 3.8 Flash	Gemini 3.5 Flash-Lite	(New!) Gemini 3.8 Live
Positioning	Complex tasks / advanced reasoning	Main workhorse / latest Flash	Fast & cost-efficient	Real-time voice
Best for	Complex reasoning · creative work	Coding · agents · knowledge work	High-volume · latency-sensitive tasks	Voice agents · live dialogue
Input	Text · Image · Video · Audio · PDF	Text · Image · Video · Audio · PDF	Text · Image · Video · Audio · PDF	Text · Image · Video · Audio
Context	1M	1M	1M	131K
Max output	64K	64K	64K	65K
Price / 1M tokens	\$2 input / \$12 output ≤200K prompt\$4 / \$18 >200K	\$0.75 input / \$3.75 output*	\$0.30 input / \$2.50 output	Text: \$0.75 / \$4.50Audio: \$3 / \$12
Launch	Feb 19, 2026	Sep 2, 2026	Jul 21, 2026	Sep 15, 2026

4) Alibaba: Qwen (general-purpose models)

Model	Main role	Launch Date
Qwen3.8-2.4T-A95B [Biggest]	Large reasoning and agent-oriented checkpoint	August 3, 2026
Qwen3.8-27B	Vision-language model; coding and multi-step tasks	August 14, 2026
Qwen3.8-Flash-Next [Smallest – A6B]	Experimental architecture focused on efficiency	August 26, 2026

Open releases: 2.4T-A95B on 12 Aug; 27B on 14 Aug 2026. Flash-Next is an architectural preview.

4) Alibaba: Qwen Specialized Models

Family	Model	Launch	Key capability	Reference
 ASR	Qwen3-ASR 1.7B / 0.6B + ForcedAligner	28 Jan 2026	52 languages/dialects · Streaming · timestamps (Thai ✓)	Official Blog · GitHub
 TTS	Qwen3-TTS 1.7B / 0.6B	21 Jan 2026	Voice Design · Voice Clone · expressive speech (No Thai Support)	Official Blog · GitHub
 Vision-Language	Qwen3-VL (2B · 4B · 8B · 32B · 30B-A3B (MoE) · 235B-A22B (MoE))	22 Sep 2025	Image/video reasoning · OCR · visual agents (Thai ✓)	Official Blog · GitHub
 Embedding / Reranker	Qwen3-VL-Embedding / Reranker (2B, 8B)	7 Jan 2026	Text + image + video → multimodal retrieval (Thai ✓)	Official Blog · GitHub

5) ByteDance: DeepSeek Models

Family	Models	Focus	Support Thai ?
General / Flagship	DeepSeek-V4.1-Flash — 552B MoE (8B input / 16B output active) [New!]	General · Agents · Coding · Multimodal	✓ Usable* (DeepSeek does not explicitly list Thai as an officially supported language.)
Reasoning	DeepSeek-R1 — 671B / 37B active	Reasoning · Math · Coding	
Vision-Language	DeepSeek-VL2 — 3.4B / 16.1B / 27.5B MoE	Image + Text · OCR · Document / Chart understanding	⚠ Not verified
Coding	DeepSeek-Coder-V2 — 16B / 236B	Code generation · Code reasoning	⚠ Not verified
Math / Formal Reasoning	DeepSeekMath-V2 — 685B · Prover-V2 — 7B / 671B	Math · Proof · Theorem reasoning	⚠ Not verified



Comparison

(as of 25 Sep 2026)

Artificial Analysis Intelligence Index [↗](#)

Artificial Analysis Intelligence Index v4.3 incorporates 10 evaluations: AA-Briefcase, GDPval-AA v2, AutomationBench-AA, Terminal-Bench 4.0, SciCode, Humanity's Last Exam, GDP.pdf, CritPt, AA-Omniscience, AA-LCR v1.1



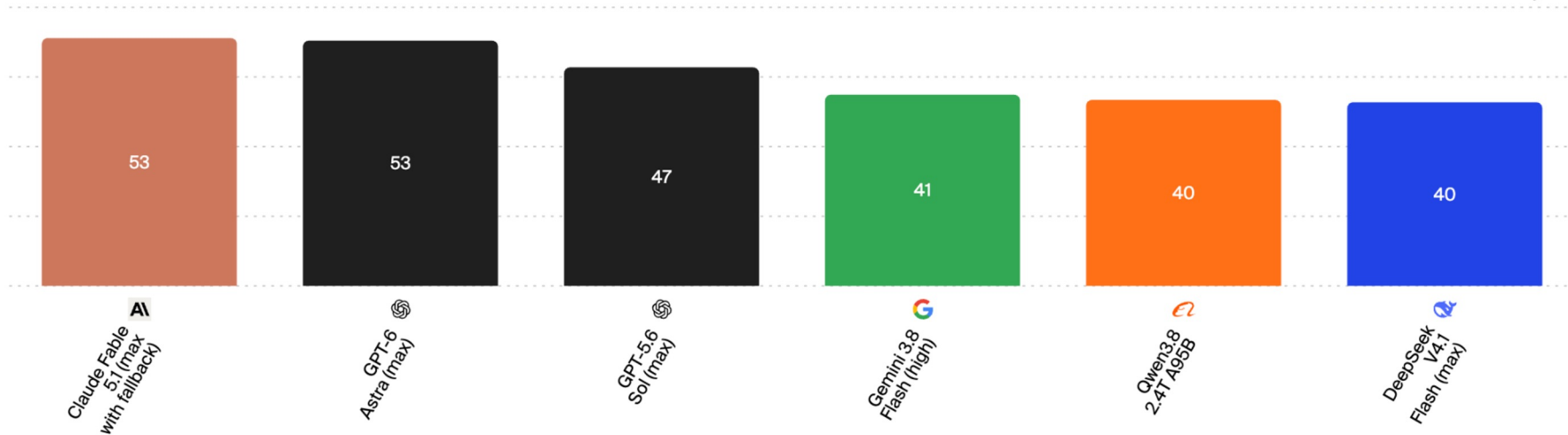
6 of 651 models



+ Add model from specific provider

Human Last Exam (HLE)

Artificial Analysis



Artificial Analysis Intelligence Index



Intelligence

- High: Fable 5.1 , GPT-6 พอๆกัน (สูงกว่า GPT-5.6 Sol)
- Medium: Gemini 3.8 Flash , DeepSeek V4.1 (Open-source)

<https://artificialanalysis.ai/>

Useful Link: <https://artificialanalysis.ai/>

Artificial Analysis Coding Agent Index ↗

Artificial Analysis Coding Agent Index v1.5 incorporates 3 benchmarks: DeepSWE v1.1, Terminal-Bench 4.0, and SWE-Atlas-QnA · Higher is better

Coding

↔

🔍

📊

Color by

Model

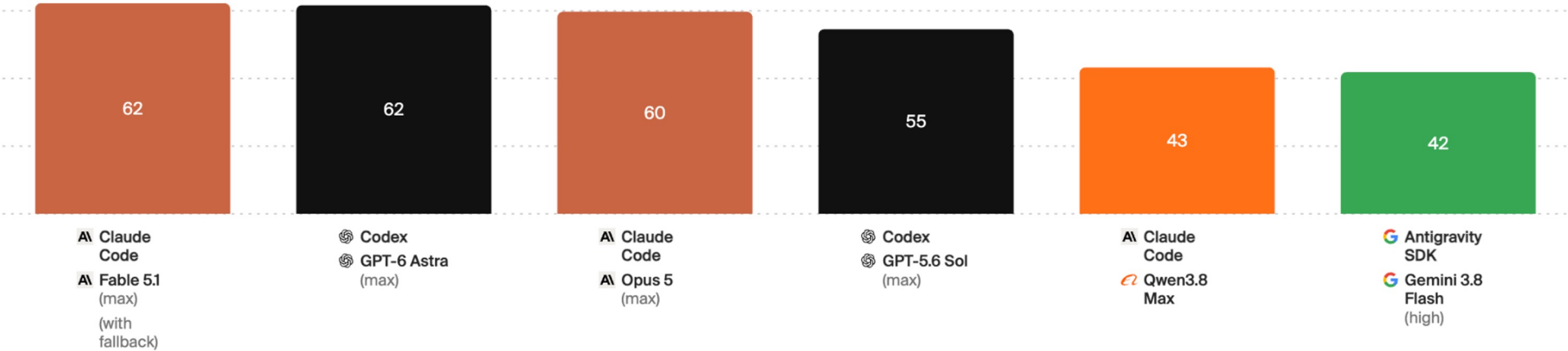
Agent

6 of 15 models

⬆

⬆

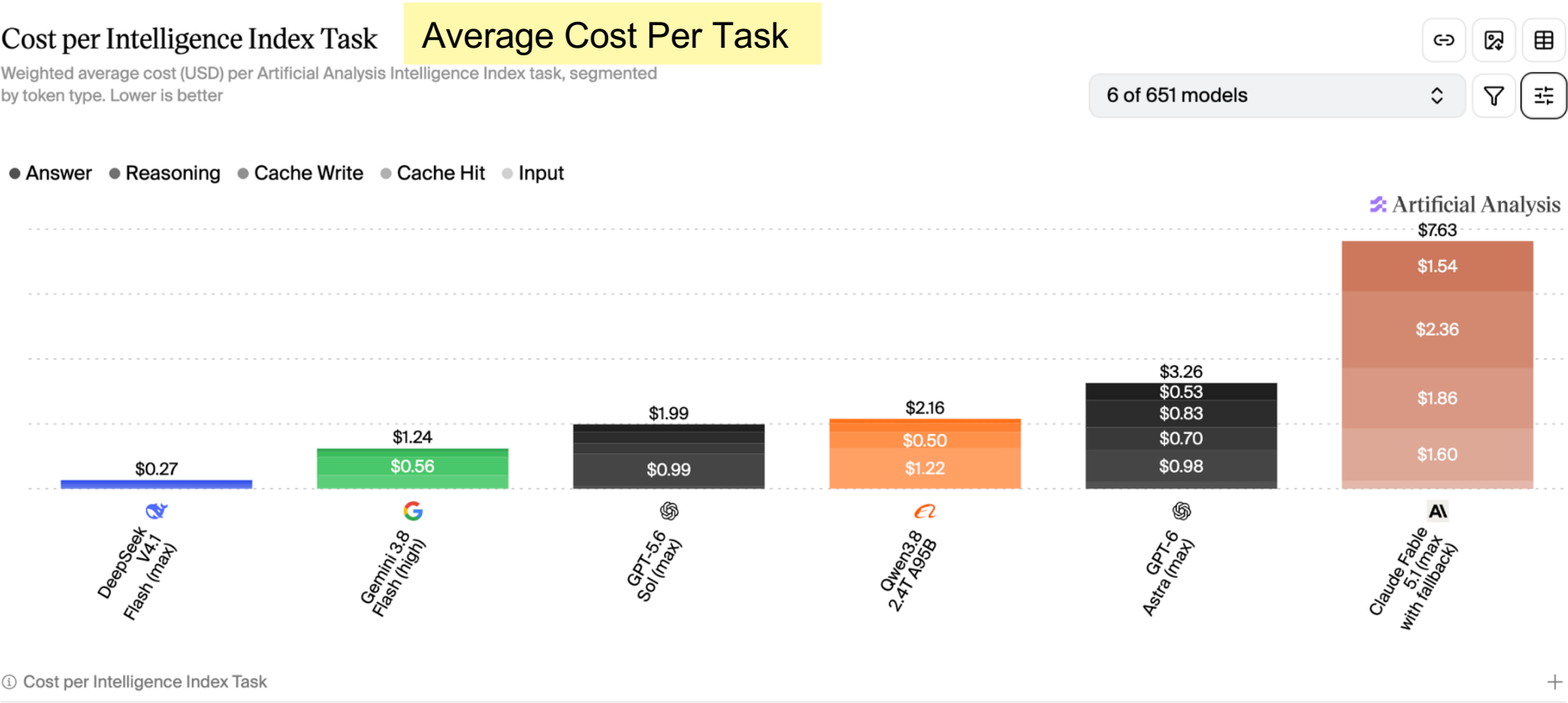
Artificial Analysis



Coding

- High: Fable 5.1, GPT-6 Astra, Opus 5
- Medium: , GPT5.6-Sol
- Low: Gemini 3.8 Flash , Qwen 3.8 (Open-source)

Useful Link: <https://artificialanalysis.ai/>



- Cost per Task โดยประมาณ
- Fable 5.1 แพงกว่า GPT-6 ประมาณ 2 เท่า
 - GPT-6 แพงกว่า GPT 5.6 ประมาณ 1.5 เท่า
 - GPT-5.6 แพงกว่า Gemini 3.8 Flash ประมาณ 1.8 เท่า
 - DeepSeek ถูกมาก

Output Speed

Speed

Output tokens per second · Higher is better

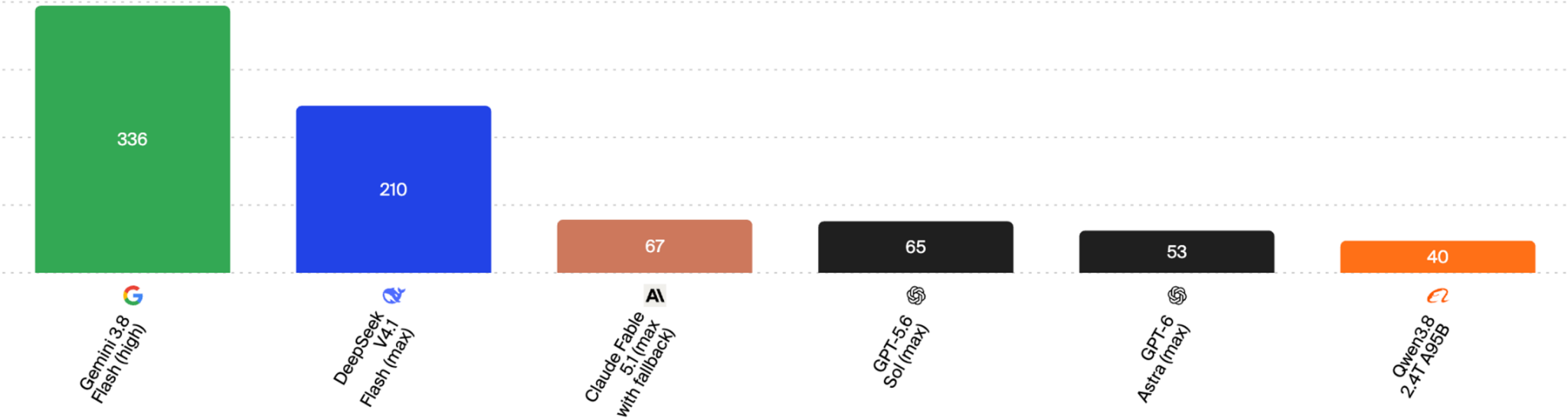


6 of 651 models



+ Add model from specific provider

Artificial Analysis



2) Prompt Engineering

LLMs Need Instructions

Poor prompt

“Analyze this.”

Better prompt

Analyze **the customer reviews** below.
Identify the three most common complaints
and return a table with category, frequency,
and one representative example.

Input, Output, Output Format

Clear task + constraints + output format → more predictable behavior

Prompt Engineering Techniques



Zero-shot
prompting



Few-shot prompting
or in-context
learning



Chain-of-thought
prompting



Tree-of-thought
prompting



Iterative
refinement



Feedback
loops



Prompt
chaining



Role-playing



Maieutic
prompting



Complexity-based
prompting

servicenow

<https://www.servicenow.com/ai/what-is-prompt-engineering.html>



Prompt types

31

1) User prompt: the specific instructions or questions a user provides to an AI system to elicit a desired response.

2) System prompt: the foundational instructions that dictate an AI's behavior.

```
1 const response = await openai.chat.completions.create({
2   model: "gpt-4o",
3   messages: [
4     {
5       "role": "developer",
6       "content": [
7         {
8           "type": "text",
9           "text": `
10            You are a helpful assistant that answers programming
11            questions in the style of a southern belle from the
12            southeast United States.
13          `
14         }
15       ]
16     },
17     {
18       "role": "user",
19       "content": [
20         {
21           "type": "text",
22           "text": "Are semicolons optional in JavaScript?"
23         }
24       ]
25     }
26   ],
27   store: true,
28 });
```

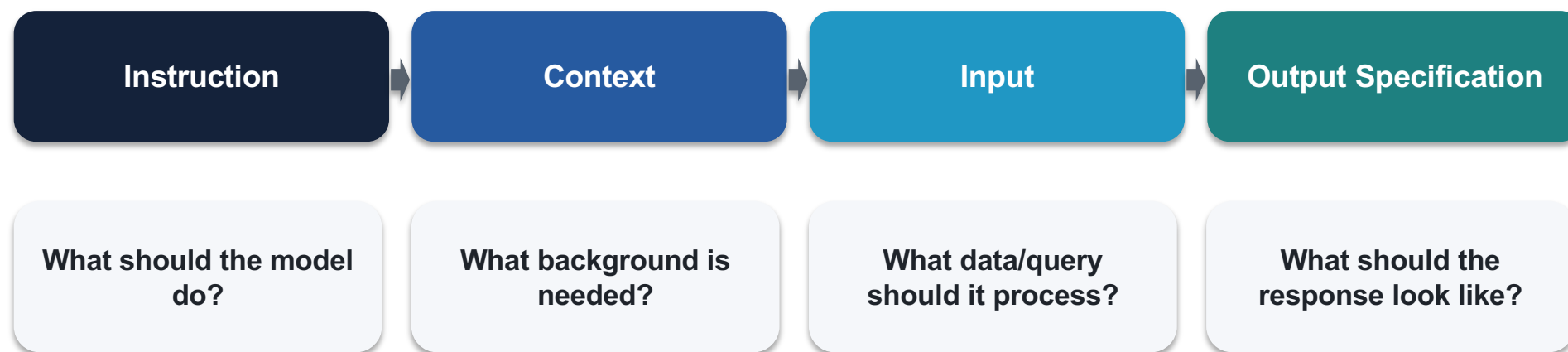
```
import anthropic

client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-3-5-sonnet-20241022",
    max_tokens=2048,
    system="You are a seasoned data scientist at a Fortune 500 company.", # <-- role
    messages=[
        {"role": "user", "content": "Analyze this dataset for anomalies: <dataset>{
    }
]

print(response.content)
```

Anatomy of a Prompt (ICIO)



A good prompt reduces ambiguity about both the task and the expected output.

+ Example 1

Instruction

Context

Input Data

Output Indicator

Classify the following text into neutral, negative, or positive.

Text: I love this food.

Sentiment:

+ Example 2

Instruction

Context

Input Data

Output Indicator

Answer the question using
information from the given context.

I love this food so much. Everything
is great ...

Question: Did the customer love the
food?

Answer:

Zero-Shot vs. Few-Shot Prompting

Zero-shot

**Classify this review as
Positive, Neutral, or Negative.**

“Good food but expensive.”

Few-shot

“Excellent food.” → Positive

“It was okay.” → Neutral

“Very slow service.” → Negative

“Good food but expensive.” → ?

Examples communicate desired behavior, format, and decision boundaries.

Structured Outputs

Natural-language output

“The review is mostly positive, although the customer mentions price concerns.”

Structured output / schema

```
{  
  "sentiment": "positive",  
  "confidence": 0.87,  
  "reason": "Good food; price concern"  
}
```

Machine-readable outputs are essential for APIs, pipelines, and agents.

Decomposing Complex Tasks (Clear Instructions)

Question: “Why did sales decrease last month?”



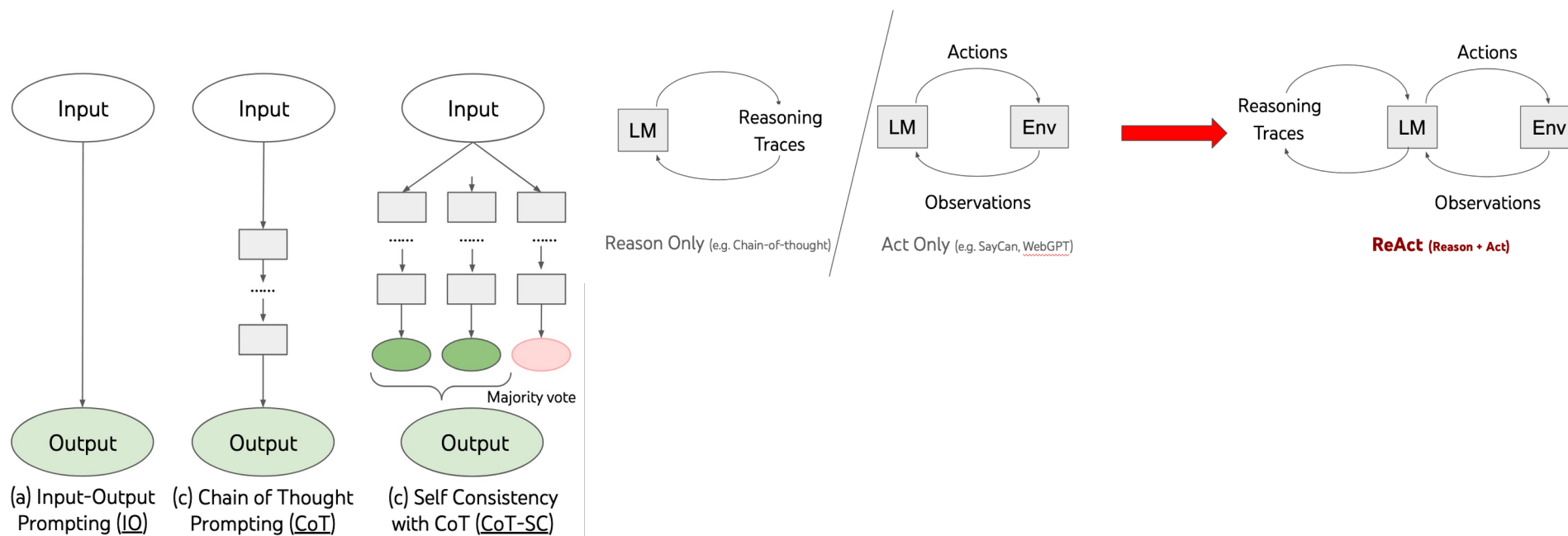
Structure the task when the problem naturally contains multiple steps.



There are many more prompting techniques.

38

- Chain-of-Thoughts (CoT)
- Self-Consistency
- ReAct



3) Retrieval Augmented Generation (RAG)

Perfect prompt ≠ access to missing knowledge

Hallucination issue → LLM cannot access a private data!

ChatGPT ▾



พืพล เวทีกุล คือใคร



พืพล เวทีกุล เป็นนักข่าวที่มีชื่อเสียงในประเทศไทย โดยเฉพาะในวงการสื่อสารมวลชน เขาเป็นที่รู้จักในฐานะผู้ประกาศข่าวทางช่อง 3 และเป็นพิธีกรรายการที่มีความนิยมในหลายรายการโทรทัศน์ เช่น รายการข่าวพิเศษต่าง ๆ และรายการสารคดีต่าง ๆ ที่ออกอากาศในช่อง 3 ของประเทศไทย



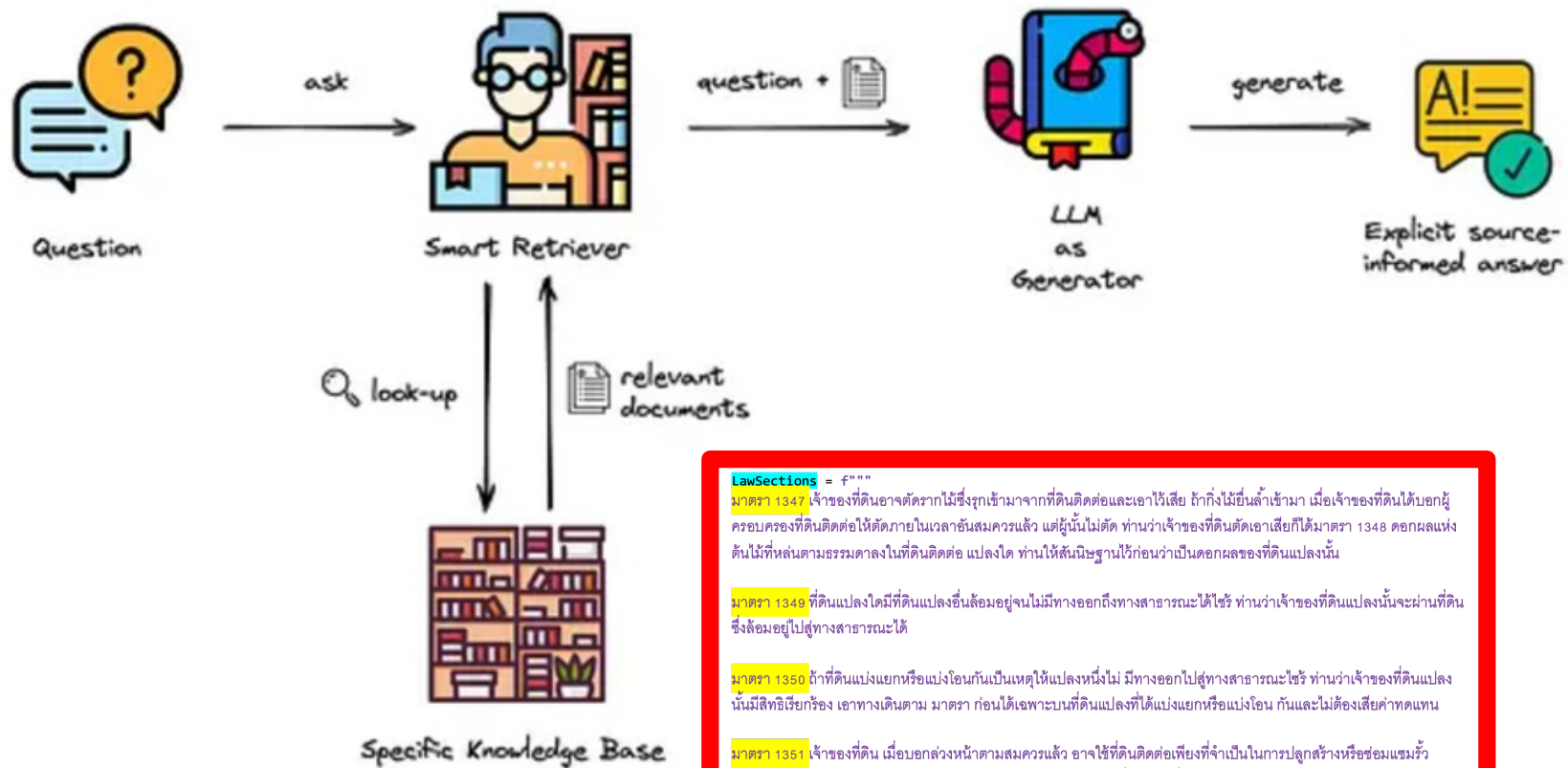
ณรงค์เดช กิตติพรานนท์ คือใคร



ณรงค์เดช กิตติพรานนท์ เป็นนักการเมืองและนักธุรกิจชาวไทยที่มีชื่อเสียง โดยเป็นสมาชิกพรรคการเมืองชั้นนำในประเทศไทย นอกจากนี้เขายังเป็นผู้บริหารในธุรกิจต่าง ๆ



Retrieval Augmented Generation (RAG)



```
LawSections = f"""
```

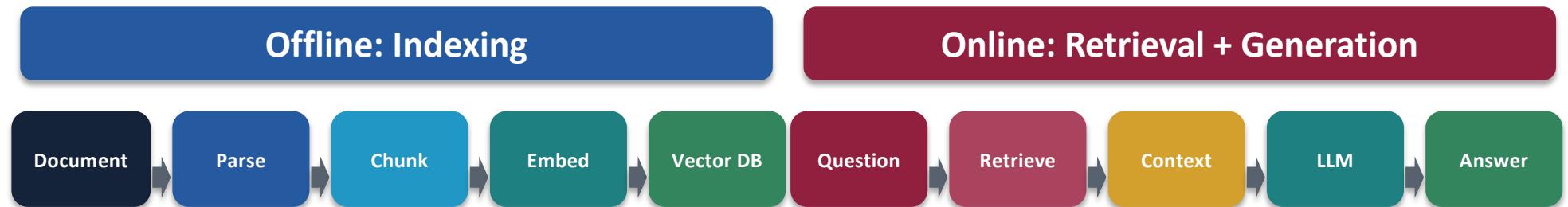
มาตรา 1347 เจ้าของที่ดินอาจตัดรากไม้ซึ่งรุกเข้ามาจากที่ดินติดต่อและเอาไปเสีย ถ้ากิ่งไม้ยื่นล้ำเข้ามา เมื่อเจ้าของที่ดินได้บอกผู้ครอบครองที่ดินติดต่อให้ตัดภายในเวลาอันสมควรแล้ว แต่ผู้ใดไม่ตัด ท่านว่าเจ้าของที่ดินตัดเอาเสียก็ได้ มาตรา 1348 ดอกผลแห่งต้นไม้ที่หั่นตามธรรมชาติลงในที่ดินติดต่อ แปลงใด ท่านให้สันนิษฐานไว้ก่อนว่าเป็นดอกผลของที่ดินแปลงนั้น

มาตรา 1349 ที่ดินแปลงใดมีที่ดินแปลงอื่นล้อมอยู่จนไม่มีทางออกถึงทางสาธารณะได้ใช้ ท่านว่าเจ้าของที่ดินแปลงนั้นจะผ่านที่ดินซึ่งล้อมอยู่ไปสู่ทางสาธารณะได้

มาตรา 1350 ถ้าที่ดินแบ่งแยกหรือแบ่งโอนกันเป็นเหตุให้แปลงหนึ่งไม่มีทางออกไปสู่ทางสาธารณะได้ใช้ ท่านว่าเจ้าของที่ดินแปลงนั้นมีสิทธิเรียกร้องเอาทางเดินตาม มาตรา ก่อนได้เฉพาะบนที่ดินแปลงที่ได้แบ่งแยกหรือแบ่งโอน กันและไม่ต้องเสียค่าทดแทน

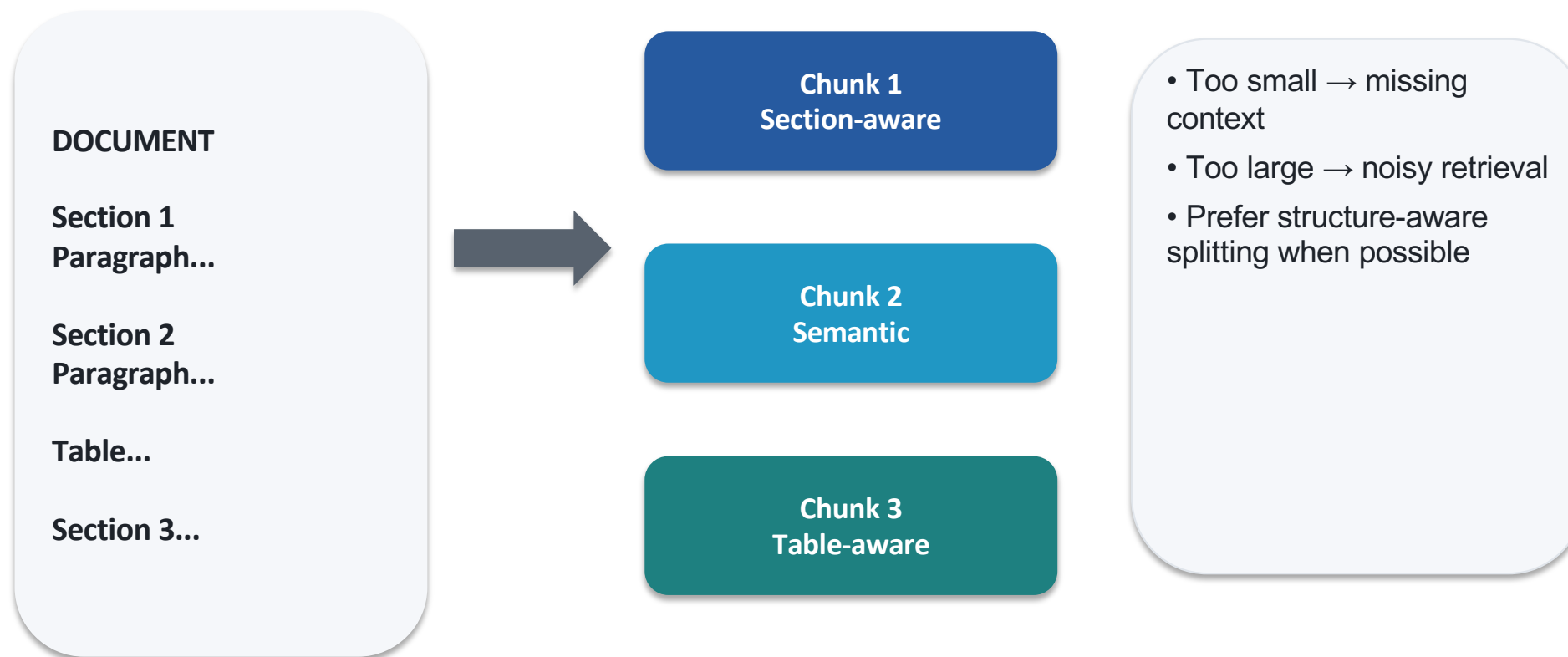
มาตรา 1351 เจ้าของที่ดิน เมื่อบอกล่วงหน้าตามสมควรแล้ว อาจใช้ที่ดินติดต่อเพียงที่จำเป็นในการปลูกสร้างหรือซ่อมแซมรั้วกำแพง หรือโรงเรือน ครัวหรือใกล้แนวเขตของตน แต่จะเข้าไปในเรือนที่อยู่ของเพื่อนบ้านข้างเคียงไม่ได้ เว้นแต่ได้รับความยินยอม

Basic RAG Architecture

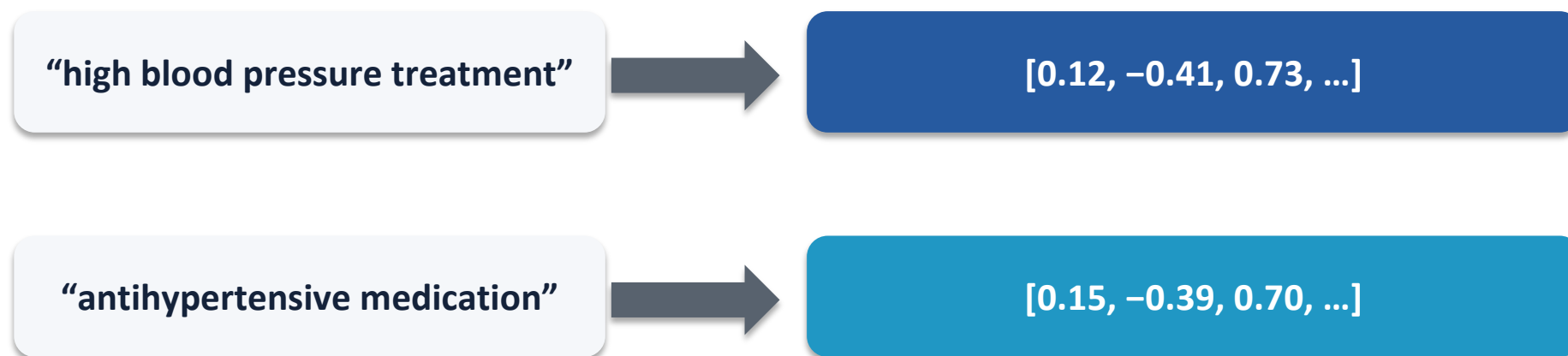


RAG changes the context at inference time — it does not require changing model weights.

Step 1: Document Parsing & Chunking

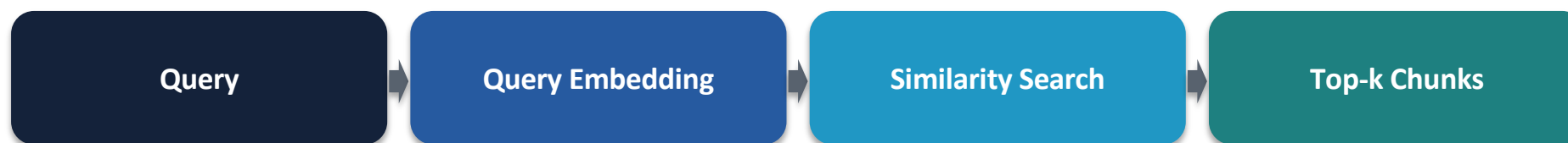


Step 2: Embeddings



Semantically similar text → nearby vectors in embedding space

Step 3: Vector Search



$$\cos(q,d) = (q \cdot d) / (||q|| \ ||d||)$$

Question: Is semantic similarity always sufficient?

Sparse vs. Dense Retrieval

Sparse / Keyword (e.g., BM25)

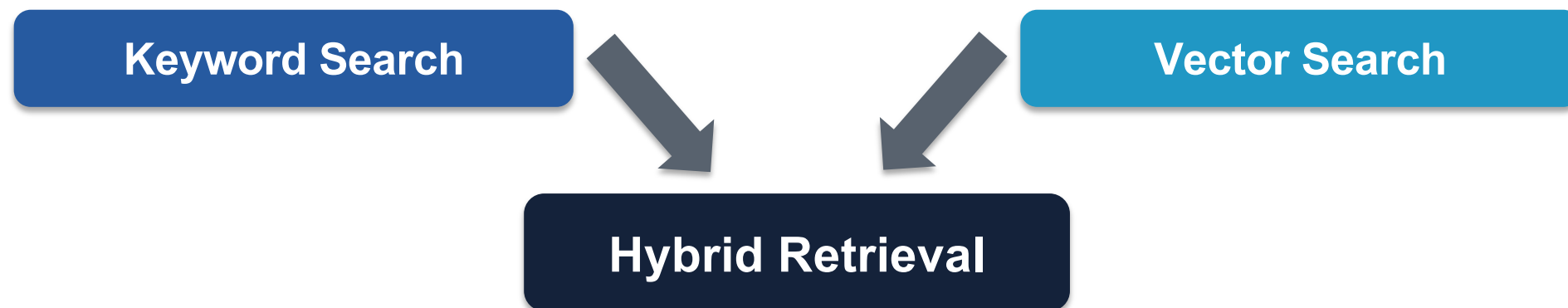
- Exact terms and rare keywords
- Names, IDs, codes
- Lexical matching

Dense / Vector

- Semantic similarity
- Paraphrases
- Conceptual matching

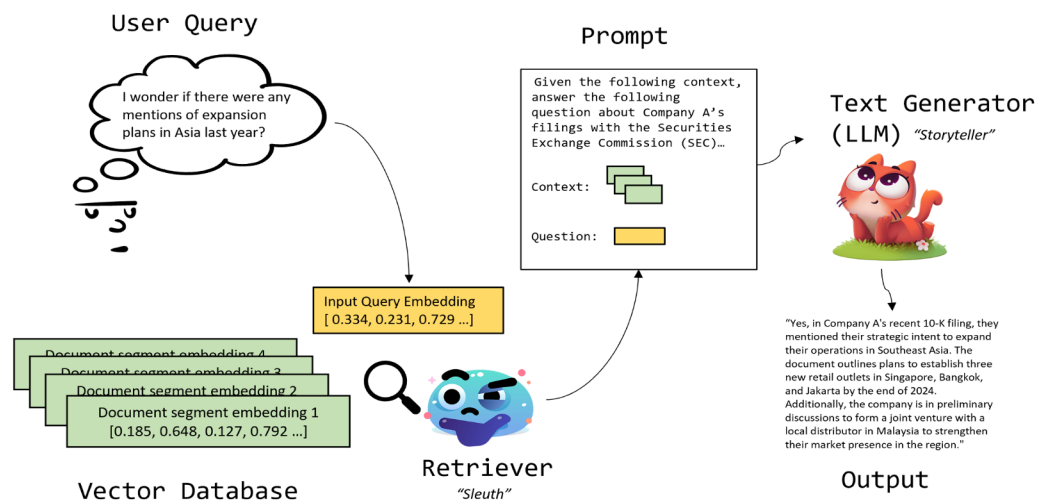
Example: “CU-2190513” may favor keyword search over semantic search.

Hybrid Search



Semantic search does not replace keyword search — they complement each other.

Retrieve → Rerank → Generate Answers

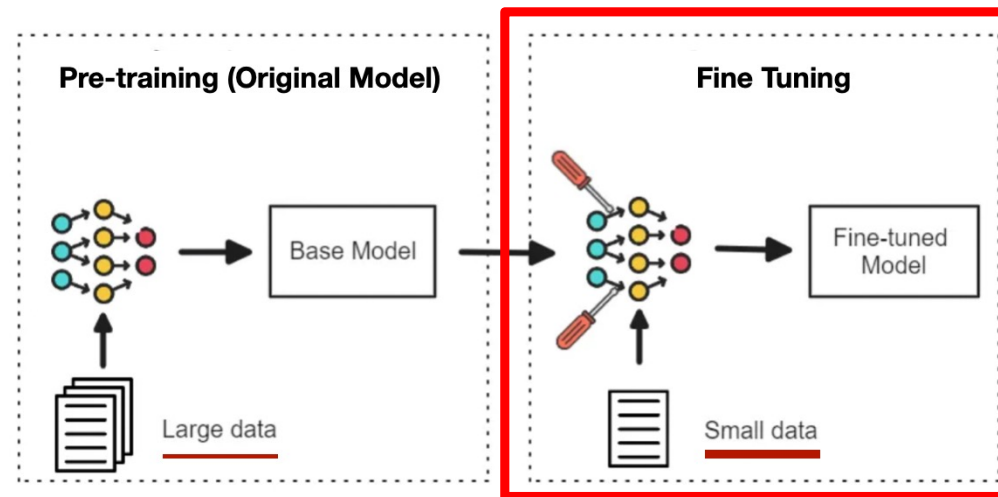


4) Fine-Tuning (briefly)

Fine-tuning

Fine-tuning is retraining a pre-trained LLM with your own domain-specific data. The goal is to make the model behave like an expert in a specific domain. It helps the model better understand specialized language, style, and tasks.

Large Language Model



Fine-tuning vs. RAG

Fine-tuning improves how the model behaves. while RAG improves what the model knows.



shutterstock.com - 680887843

Topic	Fine-tuning	RAG
Cost & Speed	Expensive to train, fast to run	Cheap to train, slower responses
Maintenance	Needs retraining when adding new data	Update data only
Knowledge Handling	Knowledge inside the model	Knowledge outside the model
Hallucination Risk	Lower in trained domain	Higher if retrieval techniques/tools is weak
Best For	Stable, repeated tasks	Changing, large knowledge

MedGemma

A collection of open models optimized for medical text and image comprehension

Read docs >

Download ▾

Accelerate the development of healthcare-based AI applications

MedGemma 1.5 4B enables developers to more effectively adapt MedGemma for applications that involve high-dimensional imaging (CT, MRI, and whole slide histopathology), longitudinal analysis of chest X-rays, anatomical localization, and other medical imaging and text functionality.

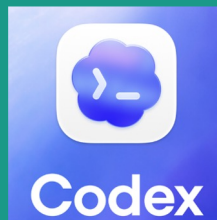
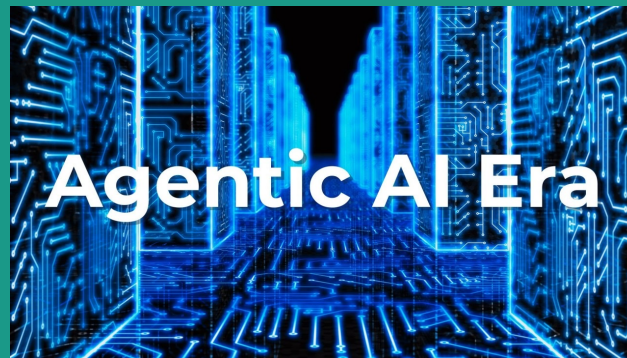
<https://deepmind.google/models/gemma/medgemma/>

WeatherNext 3

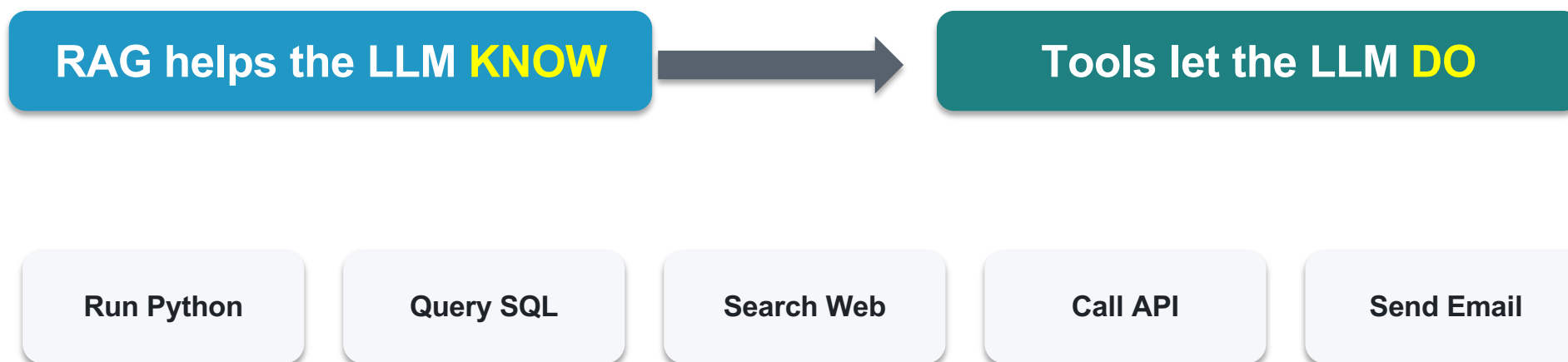
Our most advanced global weather AI model yet

Try WeatherNext 3

5) Tool Use & Agentic AI

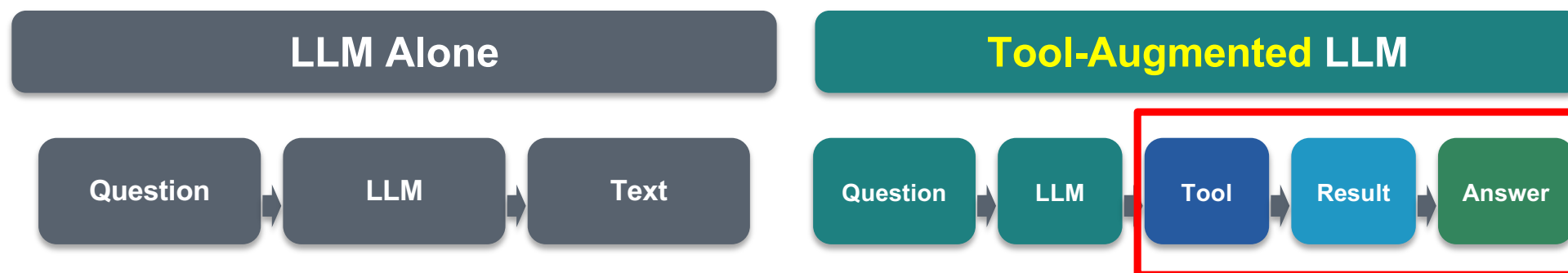


5.1 From Knowledge to Action



What if solving the task requires multiple actions and decisions?

LLM + Tools



Tools extend what the model can access, compute, and change in the external world.

Function / Tool Calling



User: "What was total revenue in Q2?"

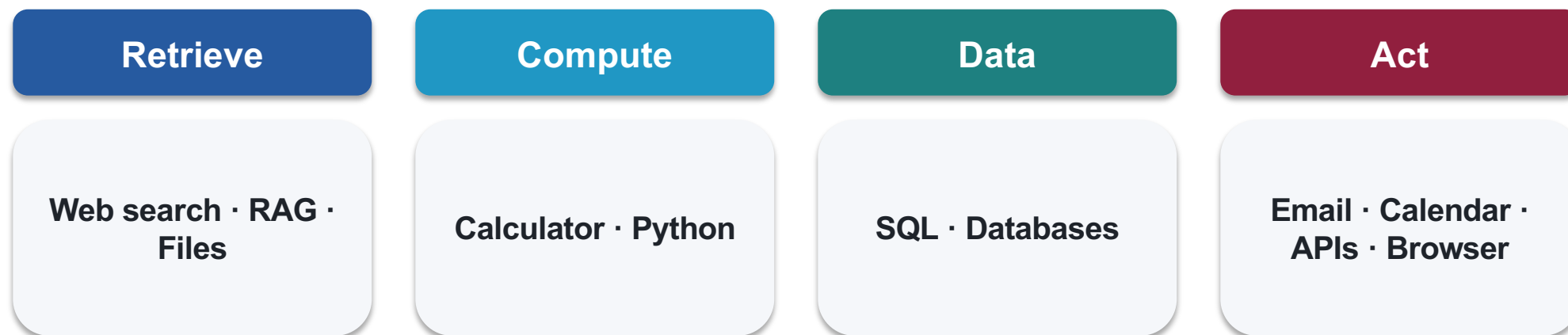
LLM requests:
query_sales(quarter="Q2",
metric="revenue")

Tool returns: 1,450,000 THB

LLM responds:
"Q2 revenue was 1.45M THB."

The model requests the tool call; the application/runtime executes it.

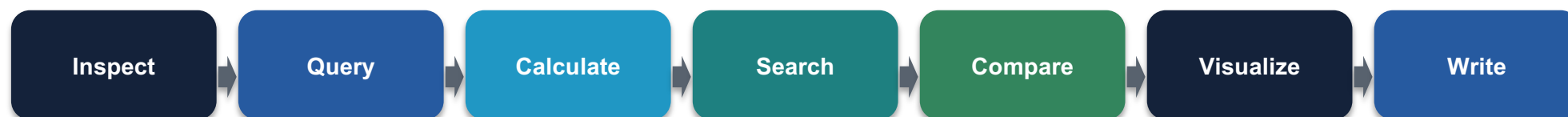
Different Types of Tools



RAG itself can be viewed as one information-retrieval capability available to an agent.

From Tool Use to Agent (Goal Oriented + Loop)

Goal: “Analyze our declining sales and prepare a report.”



Complex goals require a loop:
decide → act → observe → decide again.

Published as a conference paper at ICLR 2023

REACT: SYNERGIZING REASONING AND ACTING IN LANGUAGE MODELS

Shunyu Yao^{*1}, Jeffrey Zhao², Dian Yu², Nan Du², Izhak Shafran², Karthik Narasimhan¹, Yuan Cao²

¹Department of Computer Science, Princeton University

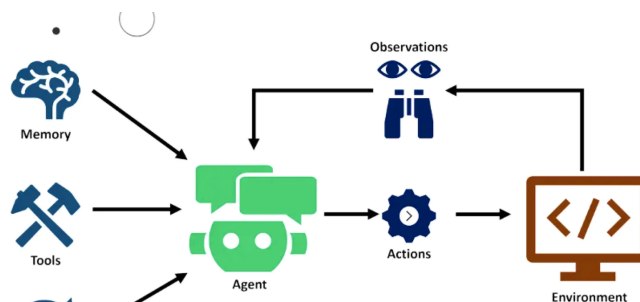
²Google Research, Brain team

¹{shunyuy, karthikn}@princeton.edu

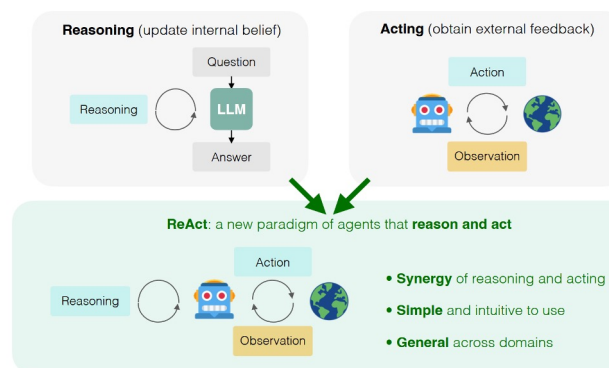
²{jeffreyzhao, dianyu, dunan, izhak, yuancao}@google.com

ABSTRACT

While large language models (LLMs) have demonstrated impressive performance across tasks in language understanding and interactive decision making, their abilities for reasoning (e.g. chain-of-thought prompting) and acting (e.g. action plan generation) have primarily been studied as separate topics. In this paper, we explore the use of LLMs to generate both reasoning traces and task-specific actions in an interleaved manner, allowing for greater synergy between the two: reasoning traces help the model induce, track, and update action plans as well as handle exceptions, while actions allow it to interface with and gather additional information from external sources such as knowledge bases or environments. We apply our approach, named ReAct, to a diverse set of language and decision making tasks and demonstrate its effectiveness over state-of-the-art baselines in addition to

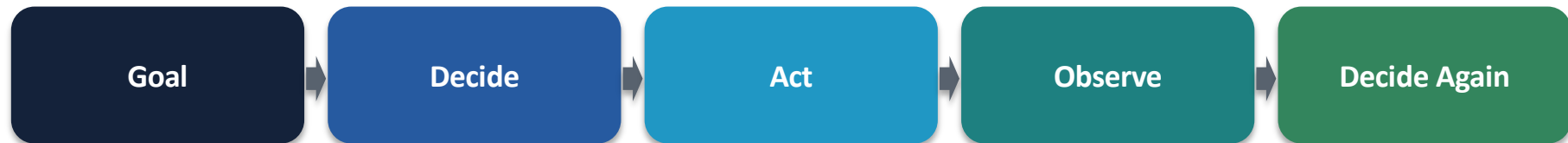


Reasoning: ReAct = Reasoning + Action

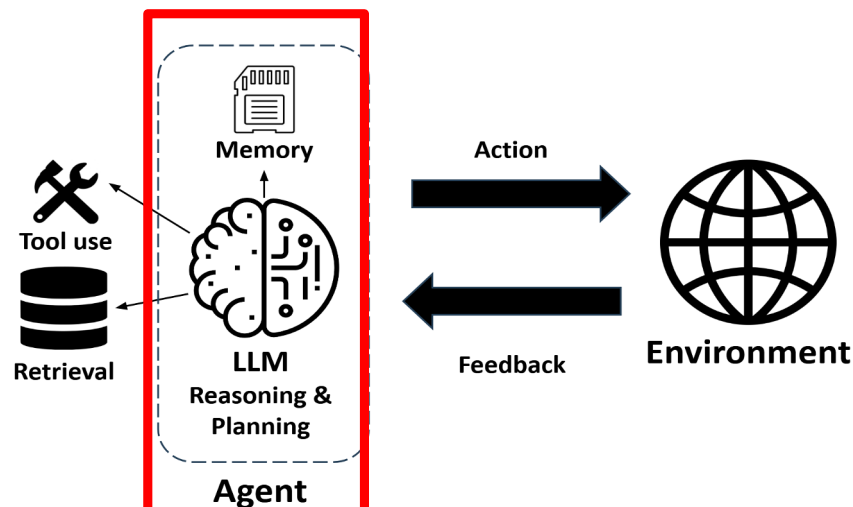


5.2 What Is an AI Agent?

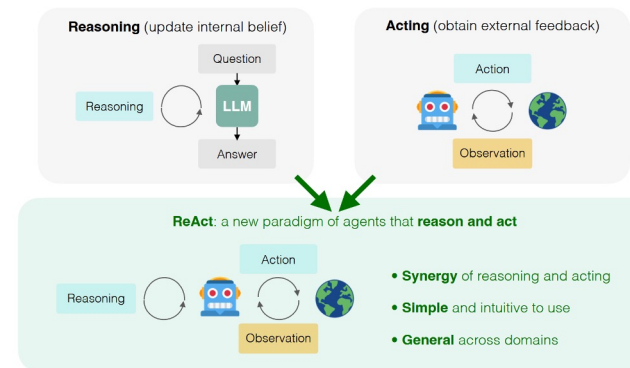
Agent = Model + Instructions + Tools + Memory/State + Loop



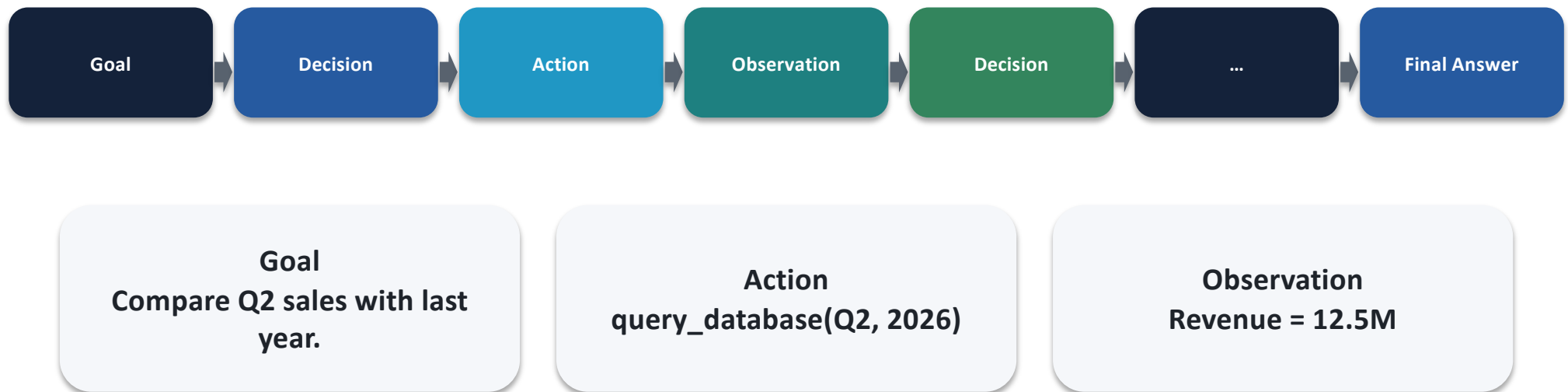
The model (**agent**) dynamically **chooses the next action** based on observations.



Reasoning: ReAct = Reasoning + Action



ReAct: Reason + Act



ReAct interleaves reasoning/decision-making with actions and observations.

Workflow vs. Agent

Workflow



- Predefined path
- Predictable / easier to test
- Lower autonomy

Agent



- Dynamic path
- Flexible / harder to test
- Higher autonomy

When NOT to Use an Agent

Every night: load CSV → aggregate → create report → email



Known steps? Use a workflow.

Use an agent when the path depends on changing intermediate results.

Single Agent vs. Multi-Agent

More agents $\neq$ better system: specialization can add cost, latency, and coordination errors.

Single Agent

Multi-Agent

Agent

Supervisor

Search

Python

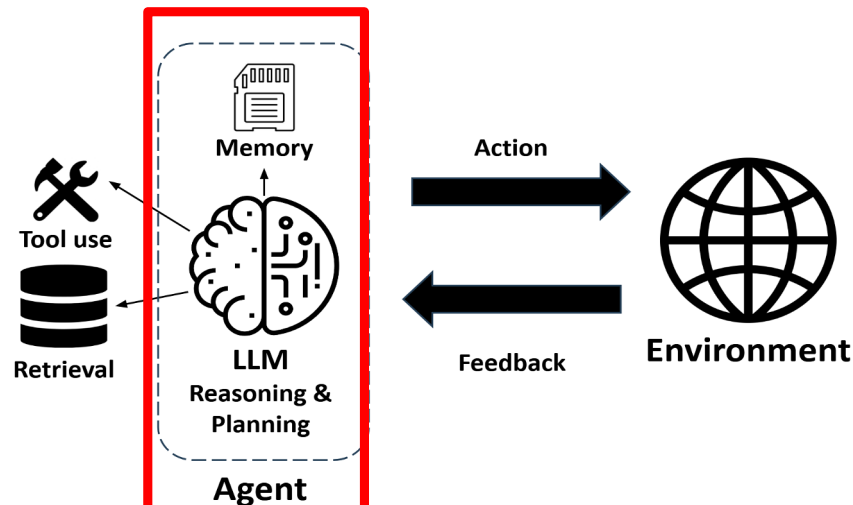
SQL

Email

Researcher

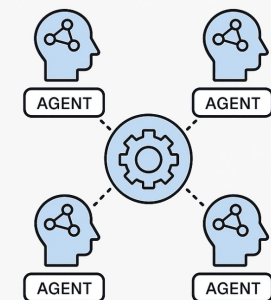
Analyst

Writer

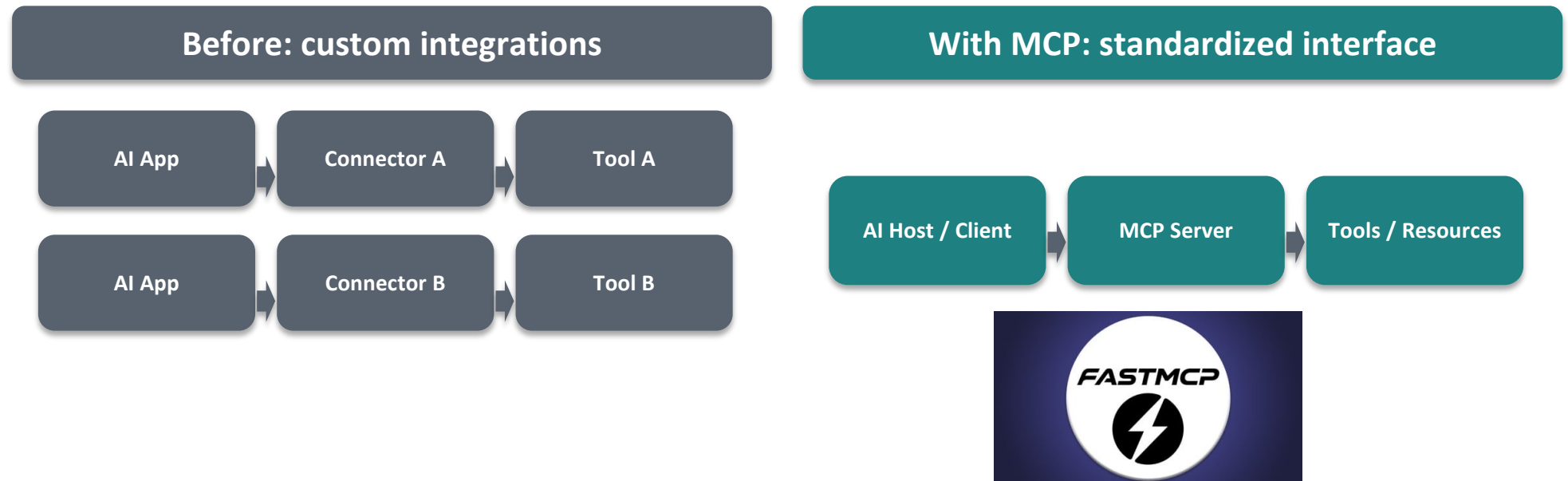


AI Agents

Agentic AI



Model Context Protocol (MCP): Connecting AI to Tools and Data



MCP standardizes how AI applications connect to external tools and context sources.

6) Context Engineering, Evaluation & Safety, Harness Engineering

We Have Built All the Pieces

Prompt

Tells the model what to do

RAG

Provides relevant knowledge

Tools

Retrieve, compute, and act

**Memory /
State**

Preserves useful information

What information should the model see at each step?

6.1 Prompt Engineering vs. Context Engineering

Prompt Engineering

How should we **instruct** the model?

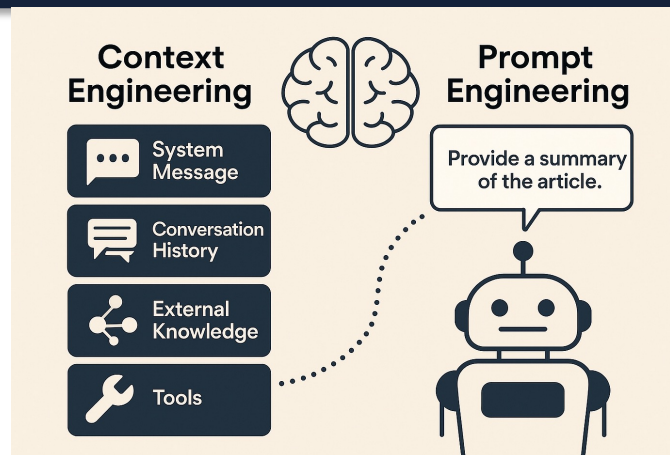
Focus: instructions, examples, constraints,
output format

Context Engineering

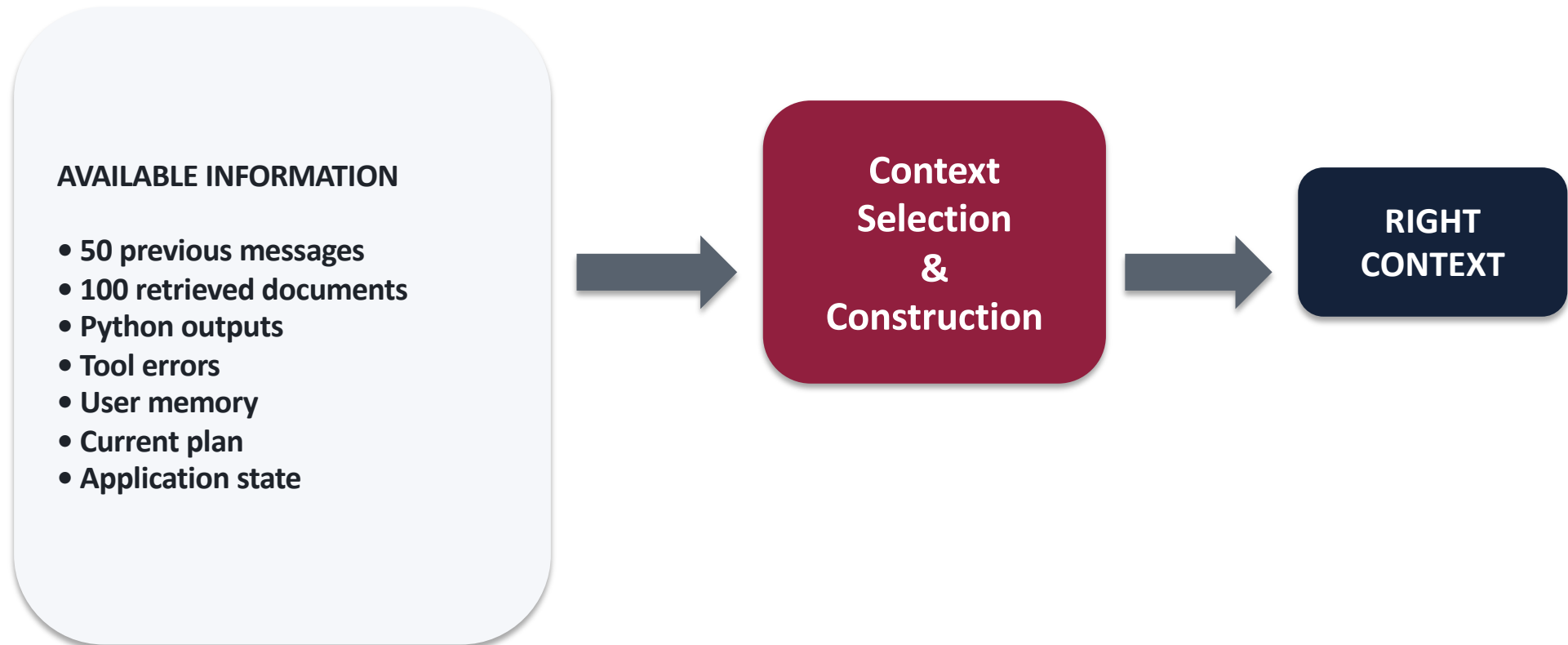
What should the model **see** to perform the task?

Prompt + history + RAG + tool results +
memory + state

Prompt Engineering $\subset$ Context Engineering



Context Engineering in an Agent



Do not send everything! — select what is relevant for the current step.

More Context $\neq$ Better Context

Maximum Context

100 documents
+ full conversation
+ all tool outputs

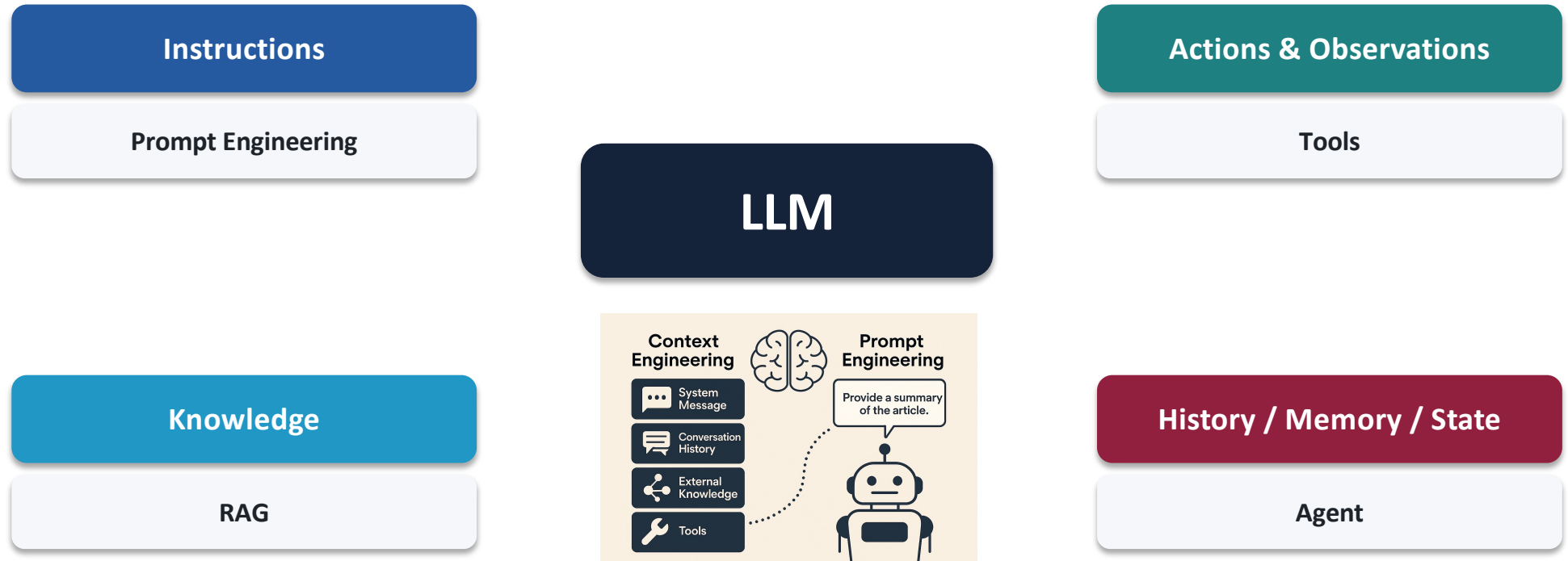
Right Context

Top relevant evidence
+ useful history
+ current state

Noise · distractors · cost · latency · context limits · buried evidence

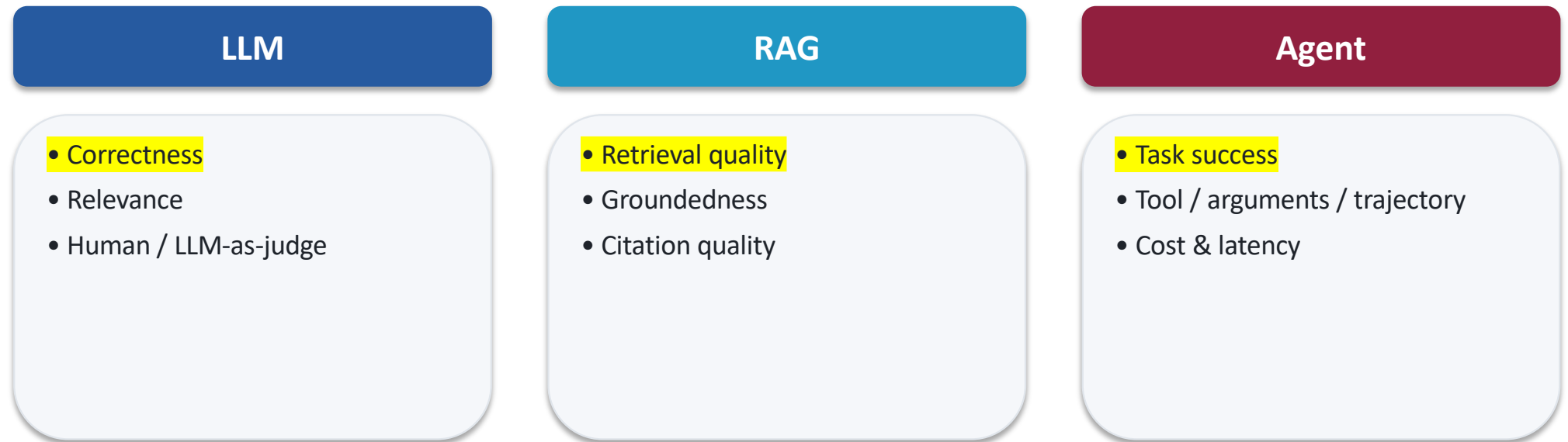
Goal: the right context, not the most context.

Context Engineering Completes the Picture



Context Engineering selects and organizes what enters the model at each step.

6.2 Evaluating LLMs, RAG, and Agents



For agents, evaluate both the final outcome and the process / trajectory.

6.3 Agentic AI Introduces New Risks

Chatbot

Wrong answer

Agent

Wrong action

- Prompt injection / indirect prompt injection
- Sensitive-data leakage
- Tool misuse / excessive agency / unauthorized actions

Guardrails & Human-in-the-Loop



- Least privilege
- Validate tool arguments
- Sandbox execution

- Approval for high-impact actions
- Trace and monitor actions
- Design safe failure modes

6.5

Prompt Engineering → What should we *tell* the AI?
Context Engineering → What should the AI *know* right now?
AI Agent → What *goal* should the AI pursue, and *what should it do next*?
Harness Engineering → How do we make the whole system *operate reliably*?

Harness Engineering

How AI agents actually work



Prompt Engineering vs Context Engineering vs AI Agent vs Harness Engineering

Prompt Engineering	→ What should we <i>tell</i> the AI?
Context Engineering	→ What should the AI <i>know</i> right now?
AI Agent	→ What <i>goal</i> should the AI pursue, and <i>what should it do next</i> ?
Harness Engineering	→ How do we make the whole system <i>operate reliably</i> ?

Layer	Main Role	Example in an AI Research Assistant
Prompt Engineering	Define what to tell the LLM and how it should respond	“Act as an expert ML reviewer. Identify three major weaknesses. For each weakness, explain the problem, provide evidence, and suggest an improvement.”
Context Engineering	Provide the right information at the right time	Supply the manuscript, reviewer comments, previous responses, relevant sections retrieved via RAG, related papers, conversation history, and tool/search results.
AI Agent	Pursue a goal and dynamically decide what to do next	Goal: “Assess whether this paper is novel and technically sound.” The agent plans what needs to be checked → reads the paper → identifies claims requiring verification → searches for related work → compares methods/results → evaluates the evidence → searches again or replans if necessary → produces the final review.
Harness Engineering	Provide the infrastructure and controls that make the agent reliable in practice	Provide Search/PDF/Python tools, tool routing, permissions, <i>sandboxing, timeouts, retries, maximum steps</i> , token/cost budgets, citation validation, error handling, and logging/observability.

AI Engineering Layers: From Prompt to Reliable Agentic System

HARNESS ENGINEERING

Tool APIs: Search / PDF / Python • Retry • Timeout • Permission • Sandbox • Logging
Validation • Max Steps • Token/Cost Budget

Prompt Engineering → What should we *tell* the AI?
Context Engineering → What should the AI *know* right now?
AI Agent → What *goal* should the AI pursue, and *what should it do next*?
Harness Engineering → How do we make the whole system *operate reliably*?

AI AGENT

🎯 **GOAL:** "Assess whether the paper is novel and technically sound."



CONTEXT ENGINEERING

Paper PDF • Reviewer Comments • Related Papers
RAG • Conversation History • Search/Tool Results • Memory

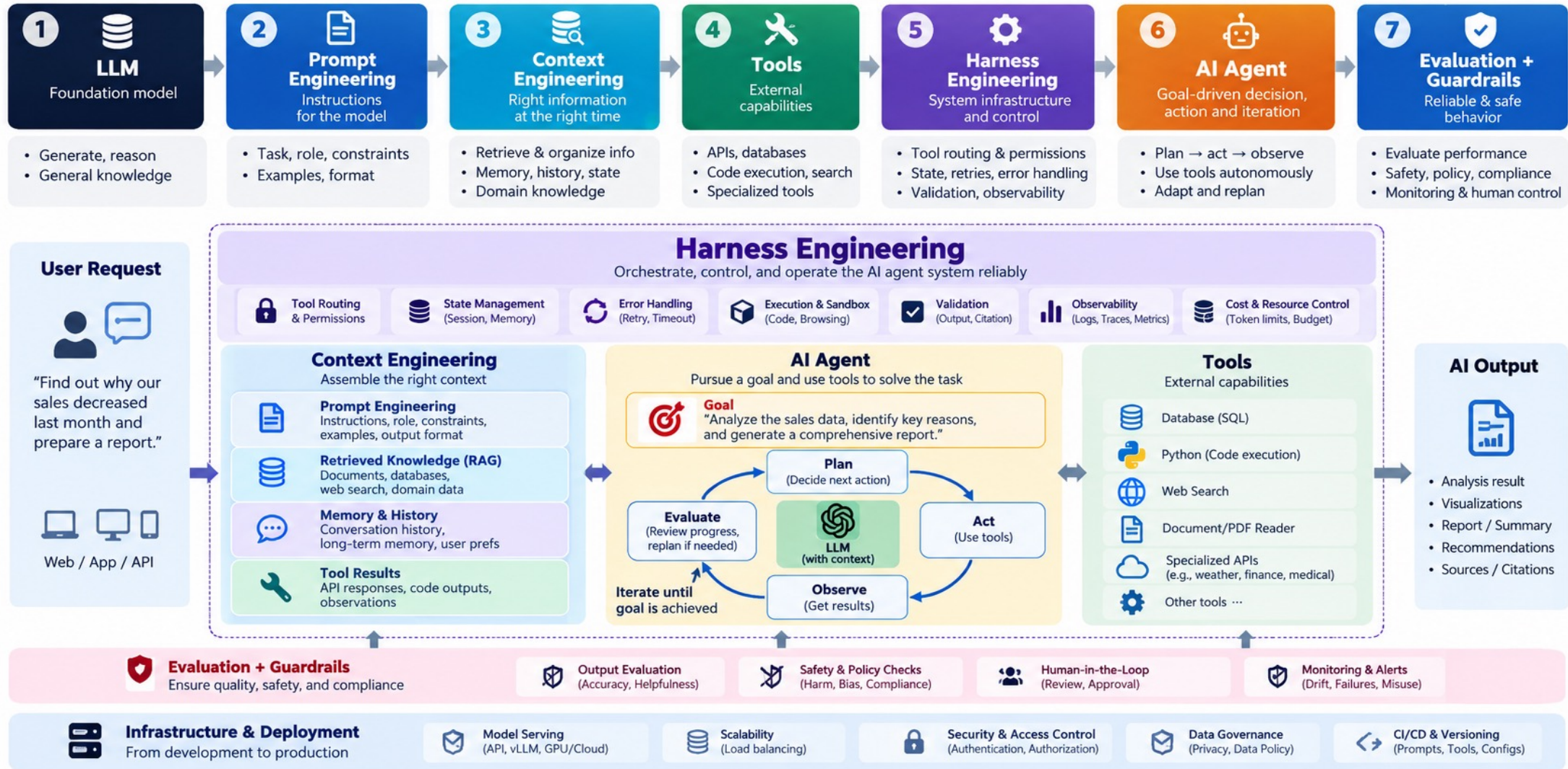
PROMPT ENGINEERING

"Act as an ML reviewer..."
"Identify major weaknesses..."
"Return evidence + suggestions..."

Prompt = what we tell the model • Context = what the model sees • Agent = goal-driven decisions/actions • Harness = reliable operation

The Big Picture: Modern GenAI System

From a foundation model to a reliable, real-world AI system



7) LLM Tools

LLM Tools

LLM Frameworks



59 Contributors 227 Used by 204 Discussions 21k Stars 2k Forks

Agentic Frameworks

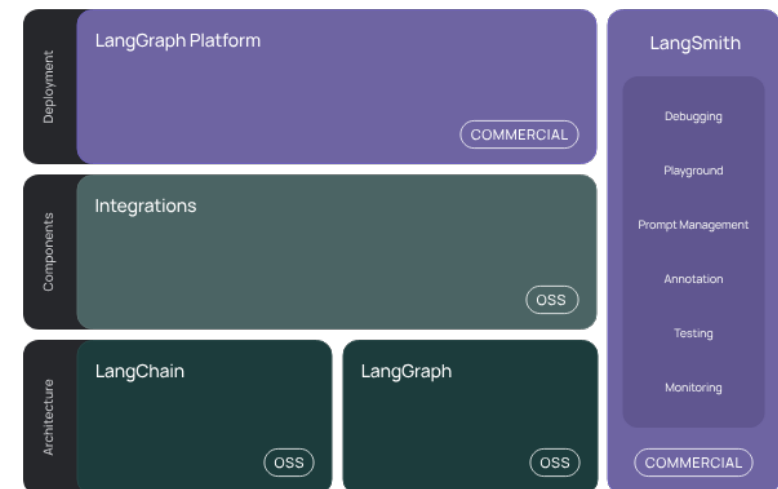
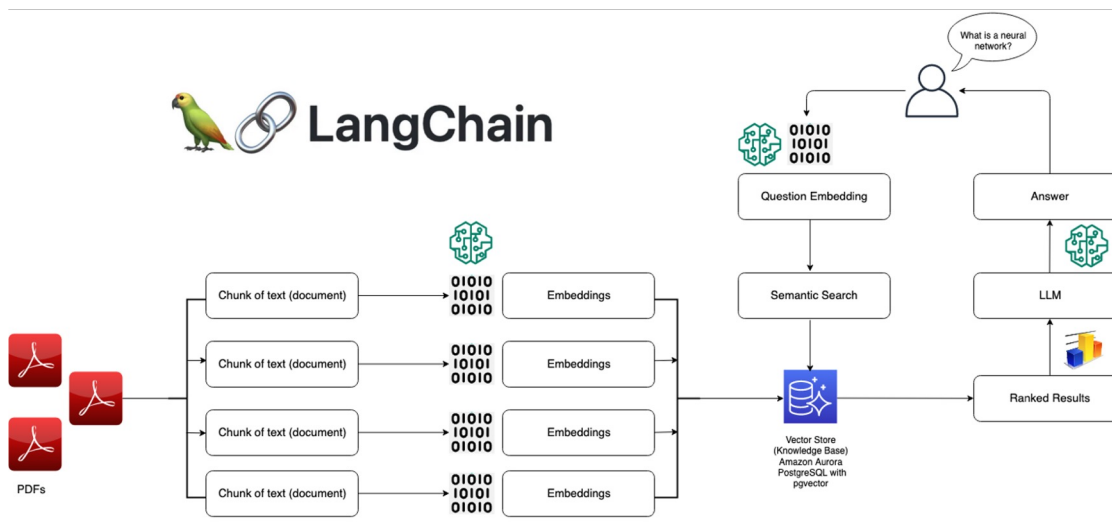


An Open-Source Programming Framework for Agentic AI
autogen@microsoft.com



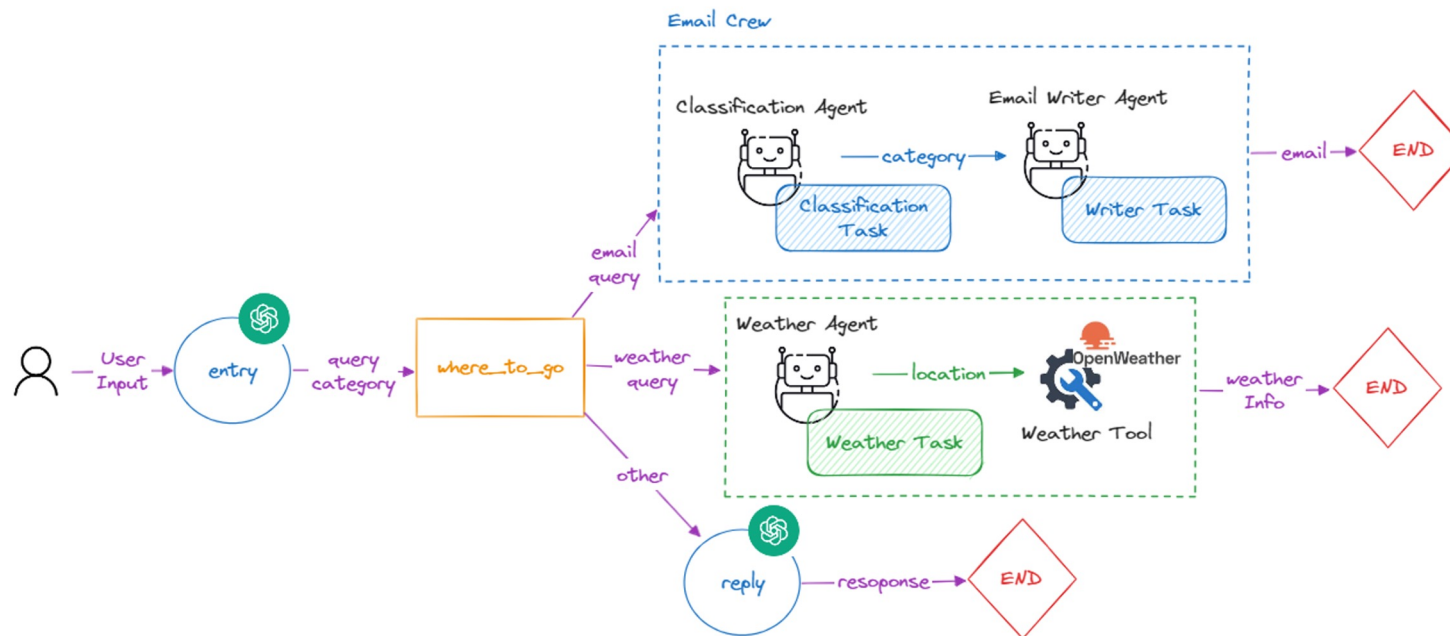
1) LangChain

LangChain is a framework that enables developers to build powerful [LLM-based applications](#) by managing external data connections, orchestrating multi-step interaction logic, and enhancing model capabilities such as context handling and tool execution, making it ideal for creating AI agents, chatbots, RAG systems, and complex automation workflows.



2) LangGraph

LangGraph is an orchestration framework for **complex agentic systems** and is more low-level and controllable than LangChain agents.



3) LangSmith (LLMOps)

Langsmith is designed with AI model debugging and orchestration in mind.

1) Observability

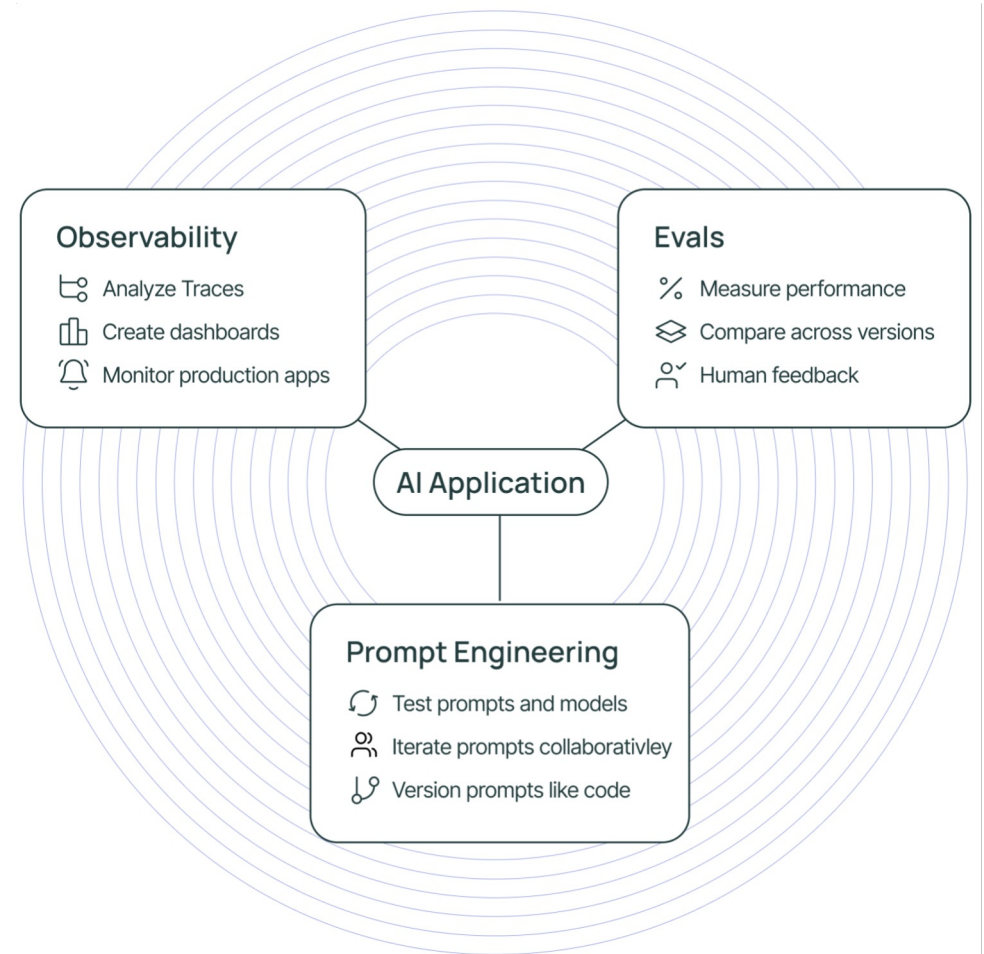
Analyze traces in LangSmith and configure metrics, dashboards, alerts based on these.

2) Evals

Evaluate your application over production traffic — score application performance and get human feedback on your data.

3) Prompt Engineering

Iterate on prompts, with automatic version control and collaboration features.



Please make sure to get all API keys before the lab.

1. gemini: <https://aistudio.google.com/app/apikey>
2. openai: <https://platform.openai.com/account/api-keys>
3. nvidia: <https://build.nvidia.com/settings/api-keys>
4. langsmith : smith.langchain.com
5. (Go to Developer (top right) → API Keys → Create API Key. Copy the key and save it securely.)
6. (optional) <https://huggingface.co/settings/tokens> (for 7_8 finetuning)



Q&A

Peerapon.v@chula.ac.th